



COMSATS University Islamabad, Pakistan

TaskHive

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Arsum Ajaz

CIIT/SP22-BCS-029/SWL

Hassaan Bin Hassan

CIIT/SP22-BCS-046/SWL

Zaheer ul Hassan

CIIT/SP22-BCS-121/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Arsum Ajaz

Hassaan Bin Hassan

Zaheer ul Hassan

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “TaskHive” was developed by **Arsum Ajaz (CIIT/SP22-BCS-029/SWL)**, **Hassaan Bin Hassan (CIIT/SP22-BCS-046/SWL)** and **Zaheer ul Hassan (CIIT/SP22-BCS-121/SWL)** under the supervision of Mr. Tariq Rafiq that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

Mr. Tariq Rafiq

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of computer science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

TaskHive is a web-based freelancing and project management platform designed to streamline collaboration between clients and freelancers through an efficient, secure, and feature-rich digital environment. Built with a focus on automation, transparency, and real-time communication. TaskHive bridges the gap between work providers and service professionals by offering an intuitive and centralized space for job posting, bidding, project execution, and secure payments.

The system enables clients to register, create job listings, manage proposals, communicate with freelancers, and track project progress seamlessly. Freelancers can browse jobs, submit bids, negotiate project requirements, deliver work, receive payments, and request withdrawals through a clean and interactive interface. For administrators, TaskHive provides a powerful dashboard to manage users, monitor payments, handle disputes, approve withdrawals, and ensure platform compliance and security. This tri-role design ensures smooth coordination between all parties while maintaining data integrity and operational control.

A key strength of TaskHive lies in its real-time collaboration capabilities, including WebSocket-based messaging, instant notifications, and workflow updates that enhance communication and reduce delays. Integrated financial handling allows secure payment processing, milestone tracking, and wallet-based earnings management. Additionally, the platform supports a transparent feedback and review system to maintain credibility and trust within the marketplace. All interactions are protected using token-based authentication, ensuring safe and controlled access.

TaskHive is built upon a scalable and modular architecture using modern technologies such as React.js, Node.js, Express.js, PostgreSQL, Auth systems, and WebSocket services for live communication. The architecture follows best-practice design principles, enabling maintainability, performance optimization, and efficient development workflows. The responsive user interface ensures accessibility across devices, providing a smooth and engaging experience for users.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor of Mr. Tariq Rafiq. Without their personal supervision, advice, and valuable guidance, completion of this project would have been doubtful. We are deeply thankful for their encouragement and continual support during this work.

We also extend our heartfelt gratitude to our families and friends who have been a constant source of support and motivation throughout this journey.

Arsum Ajaz

Hassaan Bin Hassan

Zaheer ul Hassan

Abbreviations

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DB	Database
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
REST	Representational State Transfer
RTM	Real-Time Messaging
UI	User Interface

Table of Contents

1 Introduction.....	1
1.1 Relevance to Course Modules	1
1.2 Project Background	2
1.3 Literature Review	3
1.4 Analysis from Literature Review	4
1.5 Methodology and Software Lifecycle	5
1.5.1 Rationale behind Selected Methodology:	6
2 Problem Definition.....	8
2.1 Problem Statement.....	8
2.2 Deliverables and Development Requirements.....	10
2.3 Current System	12
3 Requirement Analysis.....	14
3.1 Use Case Diagram	14
3.2 Use Case Description.....	15
3.3 Functional Requirements.....	29
3.3.1 User Registration and Login.....	29
3.3.2 Post Job.....	29
3.3.3 Browse and Bid on Jobs	29
3.3.4 Real-Time Chat.....	29
3.3.5 Receive Notifications.....	30
3.3.6 Manage Project	30
3.3.7 Submit Work.....	30
3.3.8 Process Payments.....	30
3.3.9 Request Withdrawal.....	30
3.3.10 Leave Review	31
3.3.11 Admin Manage Users	31
3.3.12 Admin Manage Jobs & Disputes	31
3.3.13 Approve Job Proposals	31
3.3.14 Approve Withdrawals.....	31
3.4 Non-Functional Requirements.....	31
3.4.1 Performance Requirements.....	32
3.4.2 Security Requirements.....	32
3.4.3 Usability Requirements	32
3.4.4 Reliability Requirements	33
3.4.5 Scalability Requirements	33
3.4.6 Maintainability Requirements.....	33
3.4.8 Compatibility Requirements.....	34
4 Design and Architecture.....	36
4.1 System Architecture	36
4.2 Data Representation.....	37
4.3 Process Flow/Representation.....	40

4.3.1 Activity Diagrams.....	40
4.4 Design Models.....	46
4.4.1 Sequence Diagram.....	46
4.4.2 Class Diagram.....	50
5 Implementation	55
5.1 Algorithm	55
5.1.1 User authentication.....	55
5.1.2 Job Posting.....	55
5.1.3 Bid Submission.....	56
5.1.4 Project Creation.....	57
5.2 External APIs.....	59
5.3 User Interface	60
5.3.1 Sign-in Screen.....	60
5.3.2 Sign-up Screen.....	61
5.3.3 Home Screen.....	62
5.3.4 Freelancer Profile Screen.....	63
5.3.5 Manage Job.....	64
5.3.6 Admin Dashboard Screen.....	65
5.3.7 Payment	66
6 Testing and Evaluation.....	68
6.1 Manual Testing.....	68
6.1.1 System Testing	69
6.1.2 Unit Testing.....	70
6.1.3 Functional Testing	72
6.1.4 Integration Testing.....	73
6.2 Tools and Technologies.....	74
6.3 Summary.....	76
7 Conclusion	78
7.1 Future Work.....	78
8 References.....	81

LIST OF TABLES

Table 1.1.1: Relevance to Course Modules	2
Table 3.2.1: User Registration and Login.....	15
Table 3.2.2: Post Job.....	16
Table 3.2.3: Browse & Bid on Jobs	17
Table 3.2.4: Real-Time Chat.....	18
Table 3.2.5: Receive Notifications.....	19
Table 3.2.6: Manage Project	20
Table 3.2.7: Submit Work.....	21
Table 3.2.8: Process Payments.....	22
Table 3.2.9: Request Withdrawal.....	23
Table 3.2.10: Leave Review	24
Table 3.2.11: Admin Manage Users	25
Table 3.2.12: Admin Manage Jobs & Disputes	26
Table 3.2.13: Approve Job Proposals	27
Table 3.2.14: Approve Withdrawals	28
Table 5.2.1: External APIs.....	59
Table 6.1.1.1: System Testing Table.....	69
Table 6.1.2.1: Unit Function Table	70
Table 6.1.3.1: Test Case Table.....	72
Table 6.1.4.1: Integration Testing Table.....	73
Table 6.2.1: Tools and Technologies Table.....	74

LIST OF FIGURES

Figure 3.1: TaskHive Use Case Diagram.....	14
Figure 4.1: TaskHive System Architecture.....	36
Figure 4.2: TaskHive ER Diagram	39
Figure 4.3: User Login and Registration Activity Diagram.....	41
Figure 4.4: Job Posting (Client) Activity Diagram	42
Figure 4.5: Bidding on Jobs (Freelancer) Activity Diagram.....	43
Figure 4.6: Project Execution Workflow Activity Diagram	44
Figure 4.7: Withdrawal Request Process Activity Diagram	45
Figure 4.8: Sequence Diagram for User Registration and Login.....	46
Figure 4.9: Job Posting (Client) Sequence Diagram.....	47
Figure 4.10: Bid Submission (Freelancer) Sequence Diagram.....	48
Figure 4.11: Project Execution Workflow Sequence Diagram.....	49
Figure 4.12: Withdrawal Request Process Sequence Diagram.....	50
Figure 4.13: Class Diagram	51
Figure 5.1: Sign-in Screen	60
Figure 5.2: Sign-up Screen	61
Figure 5.3: Home Screen	62
Figure 5.4: Freelancer Profile Screen	63
Figure 5.5: Manage Job.....	64
Figure 5.6: Admin Dashboard Screen.....	65
Figure 5.7: Payment Screen	66

Chapter 1

INTRODUCTION

1 Introduction

The rapid digitalization of the global workforce has transformed how individuals collaborate, communicate, and exchange professional services. Freelancing has become an essential pillar of the modern economy, offering flexibility to workers and cost-efficient talent acquisition for businesses. However, despite this growth, traditional freelance platforms still face limitations such as Limited support for local payment methods, high commission fee, and Global competition that disadvantages local freelancers.

TaskHive addresses these challenges by offering a modern, real-time freelance marketplace designed to streamline collaboration between clients and skilled professionals, low commission fee and support local freelancers by providing local payment methods. The system provides a seamless environment where users can post jobs, place bids, chat instantly, track project milestones, and make secure payments. Integrated WebSocket-based messaging ensures instant communication, while the structured bidding and milestone system eliminates ambiguity and enhances trust. Administrators are provided with a dedicated dashboard for monitoring platform activity, managing disputes, and ensuring fair usage across the ecosystem.

Built with the MERN stack, TaskHive is engineered for scalability, reliability, and smooth communication. Features such as JWT authentication, cloud deployment, and optimized API structures contribute to a secure and responsive user experience. By combining full-stack engineering, real-time communication technologies, and user-centric design, TaskHive demonstrates how modern software systems can elevate the freelancing experience into a faster, more transparent, and collaborative digital workflow.

1.1 Relevance to Course Modules

The development of TaskHive integrates a diverse set of concepts, tools, and methodologies studied throughout the Bachelor of Science in Computer Science (BSCS) program. The project reflects comprehensive application of theoretical knowledge and practical skills, connecting multiple course modules such as software engineering, databases, networking, web development, and HCI.

Each course module contributes to different aspects of the system as outlined below:

Table 1.1.1: Relevance to Course Modules

Course Module	Application in TaskHive
Software Engineering	Preparation of SRS, SDD, UML diagrams, requirement analysis, and structured development lifecycle.
Database Systems	PostgreSQL used for designing collections for users, jobs, bids, chats, payments, and notifications.
Web Technologies	React, Tailwind CSS, and responsive UI principles used for building interactive interfaces.
Computer Networks	Implementation of real-time communication using Web Sockets (Socket.IO).
Human-Computer Interaction (HCI)	Accessible, user-friendly client and admin dashboards designed using UX principles.
Information Security	Secure authentication using JWT, password hashing, protected routes, and API-level access control.
Project Management	Agile-based iterative development and sprint planning.

Table 1.1.1: Relevance to Course Module outlines the project not only reinforces theoretical understanding but also demonstrates practical implementation of interdisciplinary knowledge, aligning with the academic objectives of the Computer Science curriculum.

By integrating these subjects, the project provides a comprehensive demonstration of real-world application of the theoretical learning gained over eight semesters.

1.2 Project Background

The freelancing industry has grown significantly in Pakistan, with a large number of professionals offering services in areas such as software development, graphic design, content writing, and digital marketing. However, Pakistani freelancers and clients often face multiple challenges when using international freelancing platforms. These challenges include payment

and withdrawal limitations, high commission fees, lack of prioritization for local users, and intense global competition, which reduce visibility and earning potential for Pakistani freelancers.

TaskHive was conceptualized to address these challenges by providing a localized and integrated freelancing platform designed specifically for Pakistani clients and freelancers. Instead of relying on separate systems for job posting, communication, and project management, TaskHive combines all essential functionalities within a single platform. Clients can post jobs, review bids, and communicate with freelancers in real time, while freelancers can submit proposals, negotiate project terms, and manage their work efficiently. An admin dashboard is included to monitor platform activities, handle disputes, and ensure transparency and system integrity.

The platform is developed using the MERN stack, with React.js used for the frontend and Node.js with Express.js used for the backend, while PostgreSQL is utilized for data management. Real-time communication features are implemented using Socket.IO to support instant messaging and notifications. The system is designed with scalability and reliability in mind, allowing for future integration of local payment solutions such as JazzCash, EasyPaisa, and Pakistani banking systems.

Overall, TaskHive demonstrates how modern full-stack web technologies can be effectively used to build a secure, efficient, and Pakistan-focused freelancing marketplace that supports fair competition, improved collaboration, and long-term growth of the local digital economy.

1.3 Literature Review

Existing literature emphasizes the role of effective collaboration and transparency in online freelancing marketplaces. Studies on global platforms such as Upwork, Fiverr, and Freelancer.com indicate that while these systems provide basic functionalities like job posting, bidding, and messaging, they often fail to fully address the needs of freelancers and clients from developing countries such as Pakistan. Research highlights ongoing issues related to high commission fees, payment accessibility, lack of localized support, and limited visibility for emerging freelancers.

Recent studies in digital freelancing environments suggest that real-time communication and integrated workflow management significantly improve collaboration efficiency and reduce project delays. Research further indicates that platforms with structured project

management tools and milestone tracking enhance accountability and reduce disputes between clients and freelancers. However, many existing platforms either offer these features in a limited form or do not integrate them seamlessly within a single system.

Literature also points out that secure payment mechanisms and transparent review systems are critical for building trust in online marketplaces. For Pakistani users, the absence of local payment options and dependence on international gateways remains a major barrier. Additionally, studies stress the importance of user-friendly interfaces that simplify complex workflows and encourage adoption by new users.

Despite the availability of advanced features on international platforms, existing systems largely remain globally oriented and do not prioritize local market requirements. This gap in localization, combined with communication and payment challenges, highlights the need for a modern, integrated, and Pakistan-focused freelancing platform. These findings form the conceptual foundation of TaskHive, which aims to deliver transparent workflows, real-time collaboration, reduced commission rates, and future integration with local payment systems to better serve Pakistani freelancers and clients.

1.4 Analysis from Literature Review

The literature review indicates that while existing freelancing platforms offer basic functionalities such as job posting and bidding, they do not fully address the specific needs of local users, particularly in developing countries like Pakistan. Many platforms lack efficient real-time communication, transparent bidding processes, and strong administrative monitoring. As a result, users often face uncertainty in project requirements, payments, and dispute handling, which reduces trust and overall platform reliability.

1.4.1 Key Findings

- 1. Real-time communication is essential** for minimizing delays and ensuring accurate understanding of project requirements.
- 2. Transparent bidding systems** improve fairness and attract more qualified freelancers.
- 3. Secure payment workflows** reduce disputes and build trust between parties.
- 4. User interface simplicity** significantly impacts usability and retention.
- 5. Admin oversight tools** are crucial for detecting fraudulent activity and resolving conflicts.

These insights highlight the importance of combining communication technologies, clean UI/UX, and structured workflows to build a reliable freelance ecosystem.

1.4.2 Conceptualization

Based on the literature, TaskHive was designed as a complete freelancing ecosystem that integrates:

1. real-time chat and notifications
2. structured job posting and bidding
3. milestone-based project tracking
4. secure payments and withdrawals
5. comprehensive admin monitoring

TaskHive is not just a job listing platform—it is a collaborative environment that improves communication, efficiency, and overall trust between freelancers and clients.

1.5 Methodology and Software Lifecycle

TaskHive follows the Agile Software Development Lifecycle (SDLC) to accommodate continuous changes, user feedback, and iterative improvements. Given the dynamic nature of marketplace systems, Agile provides the flexibility and responsiveness needed during development.

Key Phases of the SDLC

1. Requirement Gathering

Identification of functional and non-functional requirements, including user roles, bidding system, chat module, admin controls, and payment workflows.

2. System Design

Development of UML diagrams, architectural designs, database schema, and UI prototypes to visualize system functionality.

3. Implementation

- (1) Frontend: React + Tailwind CSS
- (2) Backend: Node.js + Express
- (3) Real-time services: Socket.IO

(4) Database: PostgreSQL

4. Integration of Real-time Communication

Socket.IO integrated for instant messaging, notifications, and live updates.

5. Testing

Unit, integration, and usability testing conducted to ensure system reliability and functionality.

6. Deployment

Cloud deployment of the frontend and backend for scalability and high availability.

7. Maintenance

Ongoing updates, bug fixes, performance improvements, and feature enhancements.

1.5.1 Rationale behind Selected Methodology:

The Agile methodology was chosen due to its adaptability and iterative nature, which aligns with the evolving requirements of an online marketplace platform. Unlike traditional linear models such as Waterfall, Agile enables:

- 1) continuous feedback
- 2) incremental implementation of modules (jobs, bids, chat, payments)
- 3) rapid iteration and testing
- 4) early detection of issues
- 5) flexibility in modifying or extending features

By adopting Agile, TaskHive ensures efficient development cycles, improved product quality, and faster responsiveness to user needs.

Chapter 2

PROBLEM DEFINITION

2 Problem Definition

Although global freelancing platforms provide access to international markets, they do not adequately address the specific needs of Pakistani freelancers and clients. Users face difficulties related to payments, high commission deductions, and competitive disadvantages when competing with international freelancers. These issues limit financial growth, reduce trust, and discourage long-term participation in freelancing platforms.

TaskHive is designed to overcome these limitations by offering a localized solution that prioritizes Pakistani users, ensures affordability, and supports future integration with local financial services. The platform provides a structured, transparent, and secure environment tailored specifically for Pakistan's freelancing community.

2.1 Problem Statement

Despite the rapid expansion of online freelance platforms, several critical challenges persist in existing systems:

1. Payment and Withdrawal Issues for Pakistani Users:

Problem: Many international platforms do not fully support Pakistani payment systems. Freelancers face delays, high withdrawal fees, currency conversion losses, and limited access to funds.

Solution:

Current: TaskHive uses a wallet based internal payment tracking system to ensure transparency and accurate earnings management.

Future Goal: Integration with JazzCash, EasyPaisa, and local Pakistani banks to allow smooth payments and withdrawals in PKR, reducing dependency on international gateways.

2. High Commission Fees on Existing Platforms:

Problem: Most freelancing platforms charge up to **20% commission**, significantly reducing freelancer income and discouraging new professionals.

Solution:

Current: TaskHive is designed with a reduced commission model.

Implemented Policy: Commission will be limited to 10–12%, ensuring higher take home earnings for freelancers while maintaining platform sustainability.

3. Competitive Disadvantage for Pakistani Freelancers:

Problem: Pakistani freelancers face difficulty competing with international freelancers due to ranking algorithms, language bias, and saturated global marketplaces.

Solution:

Implemented: TaskHive prioritizes Pakistani clients and freelancers, creating a level playing field.

Local projects are highlighted, improving visibility and accessibility for Pakistani talent.

4. Lack of Localized Freelancing Ecosystem:

Problem: Existing platforms are globally oriented and do not cater to Pakistan-specific job categories, pricing norms, or communication preferences.

Solution: TaskHive is tailored to local market needs, including region-relevant job categories, pricing expectations, and communication workflows. Admin moderation ensures cultural and professional relevance.

These challenges highlight the need for a localized and integrated freelancing system that specifically addresses the needs of Pakistani freelancers and clients. Such a system should ensure fair competition, reduced commission fees, transparent workflows, and reliable project management.

TaskHive aims to build a modern, secure, and efficient freelancing platform that prioritizes Pakistani users by offering structured workflows, intuitive UI/UX, lower commission rates, and future integration with local payment methods such as JazzCash and Pakistani banks, supported by real-time communication technologies.

2.2 Deliverables and Development Requirements

The TaskHive project provides a comprehensive set of deliverables and technical requirements to ensure functionality, usability, and maintainability. The expected deliverables and development components are listed below:

2.2.1 Major Deliverables:

1. Client Interface

- 1) User registration and authentication (JWT-based).
- 2) Ability to post jobs, define budgets, and specify requirements.
- 3) Receive bids from freelancers with detailed proposals.
- 4) Real-time chat with selected freelancers.
- 5) View progress updates and approve milestone submissions.
- 6) Secure payment system for releasing funds upon completion.

2. Freelancer Interface

- 1) Create and manage freelancer profiles.
- 2) Browse and filter jobs relevant to their skill set.
- 3) Submit bids with proposed amounts, timelines, and descriptions.
- 4) Receive instant notifications for messages, job updates, and bid results.
- 5) Use real-time chat for negotiation and collaboration.
- 6) Manage ongoing projects and deliverables.

3. Real-Time Messaging Module

- 1) Socket.IO based chat system.
- 2) Read receipts, typing indicators, and instant message delivery.
- 3) Job-related notifications (new bid, bid acceptance, milestone updates).

4. Admin Dashboard

- 1) Manage user accounts (clients and freelancers).
- 2) Monitor ongoing jobs, transactions, and communication logs.
- 3) Resolve disputes between clients and freelancers.
- 4) Identify and prevent fraudulent activities.
- 5) Review platform performance analytics.

5. Database System

- 1) PostgreSQL schema for jobs, users, chats, bids, payments, milestones, and notifications.
- 2) Secure data storage with hashed passwords and controlled access.

6. Deployment

- 1) Cloud-hosted system for frontend and backend.
- 2) Scalable architecture for growing user traffic.
- 3) Continuous integration and updates.

2.2.2 Development Requirements:

Functional Requirements

- 1) User authentication and role-based access.
- 2) Job creation, editing, and deletion.
- 3) Bidding system with proposal submission.
- 4) Real-time chat and notifications.
- 5) Payment and milestone management.
- 6) Admin monitoring and dispute resolution.

Non-Functional Requirements

- 1) **Performance:** Fast loading, real-time updates, scalable design.
- 2) **Security:** Data encryption, JWT protection, secure API communication.
- 3) **Usability:** Clean, intuitive interface with responsive design.
- 4) **Reliability:** Stable connections for messaging and job updates.

5) **Maintainability:** Modular MERN architecture for easy updates.

2.3 Current System

Current international freelancing platforms such as Upwork and Fiverr are primarily designed for global markets. While they provide large-scale exposure, they exhibit several limitations for Pakistani users:

- Limited support for local payment methods
- High commission deductions
- Global competition that disadvantages local freelancers
- Lack of prioritization for Pakistani clients and projects

TaskHive improves upon these systems by focusing on affordability, accessibility, and localization. The platform emphasizes fairness, transparency, and long-term growth of Pakistan's freelancing ecosystem rather than global dominance.

Chapter 3

REQUIREMENT ANALYSIS

3 Requirement Analysis

Requirement Analysis, or Requirement Engineering, is the process of defining user expectations for new software to be built or modified. In software engineering, it is sometimes referred to loosely by names such as requirements gathering or requirements capturing. It involves gathering, analyzing, documenting, validating, and managing the needs and conditions required to meet stakeholder expectations, ensuring alignment with project goals.

3.1 Use Case Diagram

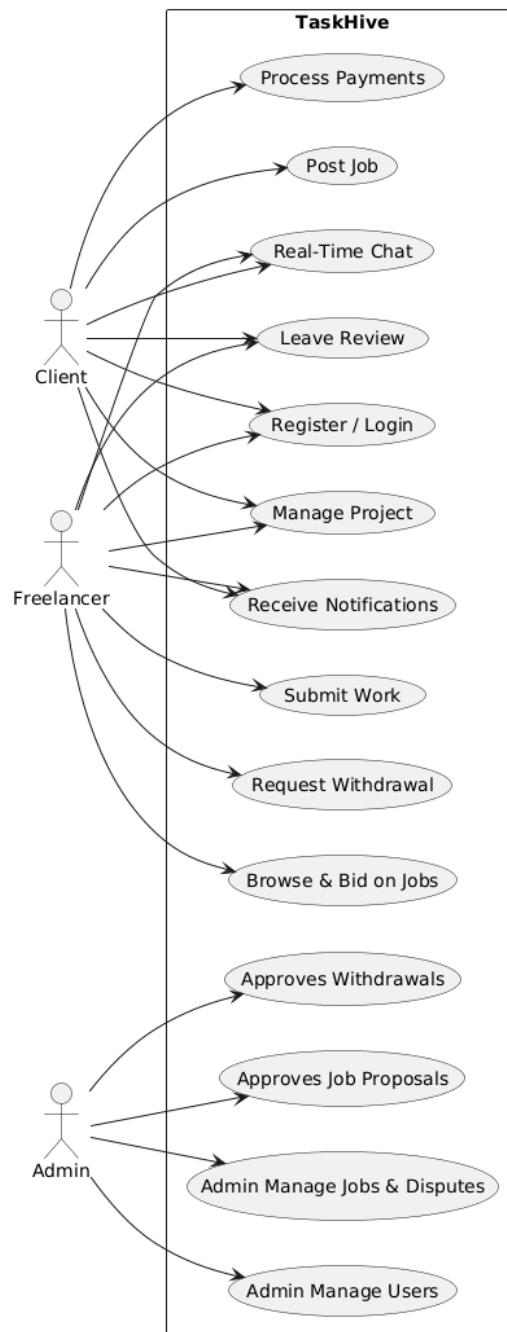


Figure 3.1: TaskHive Use Case Diagram

3.2 Use Case Description

Table 3.2.1: User Registration and login

Use Case ID	UC#01
Use Case Name	User Registration and Login
Actors	Client, Freelancer, Admin
Description	This use case describes how a user registers or logs into TaskHive using email and password.
Trigger	User clicks the “Register” or “Login” button.
Pre-condition	User must have internet access and valid credentials (for login).
Post-condition	User is successfully authenticated and redirected to the dashboard.
Normal Flow	• User selects Register/Login. • Enters credentials. • System validates credentials. • User is authenticated and redirected.
Alternative Flow	• Email already exists → system prompts to login. • Invalid fields → system shows validation error.
Exceptions	• Server down. • Authentication failed.
Business Rules	• Email must be unique. • Password must meet security rules.

Table 3.2.2: Post Job

Use Case ID	UC#02
Use Case Name	Post Job
Actors	Client
Description	Allows clients to post jobs with title, description, budget, and deadline.
Trigger	Client clicks “Post Job”.
Pre-condition	Client must be logged in.
Post-condition	Job is saved and becomes visible to freelancers.
Normal Flow	• Client opens Post Job form. • Enters required details. • System validates input. • Job is created and published.
Alternative Flow	Incomplete form → system prompts to fill required fields.
Exceptions	Server error while saving job.
Business Rules	Job must include title, budget, and deadline.

Table 3.2.3: Browse & Bid on Jobs

Use Case ID	UC#03
Use Case Name	Browse & Bid on Jobs
Actors	Freelancer
Description	Freelancers browse available jobs and place bids with proposed amount and delivery time.
Trigger	Freelancer clicks “Browse Jobs”.
Pre-condition	Freelancer must be logged in.
Post-condition	Bid is submitted and visible to the client.
Normal Flow	• Freelancer views job list. • Filters/searches jobs. • Opens job details. • Submits bid.
Alternative Flow	Freelancer withdraws or edits a bid before client approval.
Exceptions	Job removed during bidding.
Business Rules	Freelancers cannot bid on their own posted jobs.

Table 3.2.4: Real-Time Chat

Use Case ID	UC#04
Use Case Name	Real-Time Chat
Actors	Client, Freelancer
Description	Enables communication between client and freelancer using WebSocket-based chat.
Trigger	Either user opens a chat window.
Pre-condition	Both must be assigned to the same job.
Post-condition	Message delivered and stored.
Normal Flow	• User opens chat. • Sends message. • System delivers message instantly.
Alternative Flow	User sends attachment.
Exceptions	• Connection lost. • WebSocket server offline.
Business Rules	Chat only available for active jobs.

Table 3.2.5: Receive Notifications

Use Case ID	UC#05
Use Case Name	Receive Notifications
Actors	Client, Freelancer
Description	Users receive real-time notifications related to job updates, chats, bids, and payments.
Trigger	Any system event requiring a notification.
Pre-condition	User must be logged in.
Post-condition	Notification displayed in UI.
Normal Flow	• System detects event. • Pushes notification to user.
Alternative Flow	Notification stored if user offline.
Exceptions	WebSocket failure.
Business Rules	Notifications must be timestamped.

Table 3.2.6: Manage Project

Use Case ID	UC#06
Use Case Name	Manage Project
Actors	Client, Freelancer
Description	Both parties manage project milestones, deadlines, and task updates.
Trigger	User clicks “Manage Project”.
Pre-condition	Job must be accepted and active.
Post-condition	Project status updated.
Normal Flow	• User views project details. • Updates milestones or checks progress.
Alternative Flow	Freelancer requests deadline extension.
Exceptions	Missing milestone data.
Business Rules	Only assigned users may manage the project.

Table 3.2.7: Submit Work

Use Case ID	UC#07
Use Case Name	Submit Work
Actors	Freelancer
Description	Freelancer uploads final deliverables for client review.
Trigger	Freelancer clicks “Submit Work”.
Pre-condition	All required project milestones must be completed.
Post-condition	Work sent to client for approval.
Normal Flow	• Freelancer uploads files. • System validates file type/size. • Submission sent to client.
Alternative Flow	Freelancer edits submission.
Exceptions	Upload fails due to server/storage error.
Business Rules	Only the assigned freelancer can submit work.

Table 3.2.8: Process Payments

Use Case ID	UC#08
Use Case Name	Process Payments
Actors	Client
Description	Client pays for the completed project using secure payment gateway.
Trigger	Client clicks “Pay Now”.
Pre-condition	Work must be reviewed and accepted.
Post-condition	Payment processed and recorded.
Normal Flow	• Client opens payment page. • Enters payment details. • System processes transaction.
Alternative Flow	Payment fails → retry option.
Exceptions	Payment gateway timeout.
Business Rules	All transactions are logged and encrypted.

Table 3.2.9: Request Withdrawal

Use Case ID	UC#09
Use Case Name	Request Withdrawal
Actors	Freelancer
Description	Freelancer requests payout of available earnings to bank or wallet.
Trigger	Freelancer clicks “Withdraw”.
Pre-condition	Freelancer must have sufficient balance.
Post-condition	Withdrawal request sent to admin.
Normal Flow	• Freelancer enters amount. • System validates balance. • Request forwarded to admin.
Alternative Flow	Freelancer edits or cancels withdrawal request.
Exceptions	Invalid bank details.
Business Rules	Minimum withdrawal limit applies.

Table 3.2.10: Leave Review

Use Case ID	UC#10
Use Case Name	Leave Review
Actors	Client, Freelancer
Description	Users rate each other and write reviews after project completion.
Trigger	User clicks “Leave Review”.
Pre-condition	Project must be completed.
Post-condition	Review saved and added to profile.
Normal Flow	• User enters rating & comment. • System validates input. • Review submitted.
Alternative Flow	User edits the review later (if allowed).
Exceptions	Review exceeds character limits.
Business Rules	Only verified project partners can review each other.

Table 3.2.11: Admin Manage Users

Use Case ID	UC#11
Use Case Name	Admin Manage Users
Actors	Admin
Description	Admin can view, block, edit, or delete platform users.
Trigger	Admin opens User Management panel.
Pre-condition	Admin must be authenticated.
Post-condition	User account updates are applied.
Normal Flow	• Admin views user list. • Selects a user. • Applies action (block, delete, edit).
Alternative Flow	Admin filters users by status.
Exceptions	User record missing.
Business Rules	Admin actions must be logged.

Table 3.2.12: Admin Manage Jobs & Disputes

Use Case ID	UC#12
Use Case Name	Admin Manage Jobs & Disputes
Actors	Admin
Description	Admin resolves disputes, edits job status, and handles fraudulent activities.
Trigger	Admin opens Job/Dispute panel.
Pre-condition	Admin authenticated.
Post-condition	Job/dispute updated.
Normal Flow	• Admin views pending disputes. • Reviews evidence. • Updates status.
Alternative Flow	Admin escalates dispute.
Exceptions	Missing dispute evidence.
Business Rules	Admin must respond within system-defined SLA.

Table 3.2.13: Approve Job Proposals

Use Case ID	UC#13
Use Case Name	Approve Job Proposals
Actors	Admin
Description	Admin reviews and approves job posting proposals before publishing.
Trigger	New job proposal submitted.
Pre-condition	Valid job details exist.
Post-condition	Job approved or rejected.
Normal Flow	• Admin views proposals. • Approves or rejects.
Alternative Flow	Sends job back for revision.
Exceptions	Invalid job data.
Business Rules	Only approved jobs appear publicly.

Table 3.2.14: Approve Job Proposals

Use Case ID	UC#14
Use Case Name	Approve Withdrawals
Actors	Admin
Description	Admin reviews and approves freelancer withdrawal requests.
Trigger	Withdrawal request submitted.
Pre-condition	Freelancer has sufficient balance.
Post-condition	Withdrawal processed.
Normal Flow	• Admin reviews request. • Approves payment.
Alternative Flow	Rejects request with reason.
Exceptions	Invalid payment details.
Business Rules	All withdrawal approvals logged.

3.3 Functional Requirements

Functional requirements define how the TaskHive Freelancing Marketplace System should operate, describing system behaviors, user interactions, and conditions under which the system performs specific actions. These requirements ensure that clients, freelancers, and administrators can efficiently use the platform for posting jobs, bidding, communication, project management, and payment processing. The following subsections outline the core functional modules of TaskHive.

3.3.1 User Registration and Login

The system must provide a secure and reliable authentication mechanism for Clients, Freelancers, and Admins. Users should be able to register using their email and password, and the system must validate input fields during registration and login. Session management must be performed using secure authentication tokens. Incorrect login attempts must trigger error messages, and unauthorized access must be prevented through password complexity rules and validation checks.

3.3.2 Post Job

The system must allow Clients to create and publish job posts. The job posting module must include fields such as title, description, budget, category, skills required, and deadline. The system must validate all mandatory fields before saving the post. Once created, the job should immediately become visible to eligible freelancers. The system must prevent posting incomplete or invalid job data.

3.3.3 Browse and Bid on Jobs

The system must allow Freelancers to browse available jobs using filters such as category, skills, price range, and deadlines. Freelancers should be able to view full job details and place bids containing bid amount, estimated delivery time, and proposal text. The system must ensure that freelancers cannot bid on their own jobs and must validate all bid inputs before submission. The client should be notified when a new bid is received.

3.3.4 Real-Time Chat

The system must provide a real-time messaging feature to allow Clients and Freelancers to communicate once a bid is approved or a project is in progress. The chat module should support text messaging, message timestamps, and file attachments. WebSocket or real-time communication protocols must be used to deliver messages instantly. The system must store chat history for future reference and display unread message indicators.

3.3.5 Receive Notifications

The system must deliver real-time notifications to users regarding key activities such as new bids, job updates, message alerts, payment confirmations, and withdrawal approvals. Notifications must appear instantly and be accessible in a notification panel. If a user is offline, notifications must be stored and delivered once the user reconnects. Each notification must include a timestamp and clear message content.

3.3.6 Manage Project

The system must provide a centralized module for managing active projects. Clients and Freelancers must be able to track milestones, deadlines, submitted work, and project progress. The system must allow updates to project status, milestone approvals, and progress tracking. Only assigned users should be able to view and modify the project details. The system should log all activity for transparency.

3.3.7 Submit Work

The system must allow Freelancers to upload and submit final deliverables for reviewed projects. The module must support file uploads, text notes, and version updates. The system should validate file types and size limits before submission. Once submitted, the client must be notified and allowed to approve or request revisions. Submitted work must be stored securely.

3.3.8 Process Payments

The system must provide a secure method for Clients to pay for completed jobs. Payments should be processed using an integrated payment gateway, ensuring encryption and transaction logging. The system must validate payment details and provide confirmation messages for successful transactions. Failed payments must display appropriate errors and allow retry attempts. All payments must be recorded for accounting.

3.3.9 Request Withdrawal

The system must allow Freelancers to request withdrawal of their earned funds. Freelancers must be able to enter withdrawal amounts and bank or wallet details. The system must validate available balances before allowing withdrawal requests. Once submitted, the request must be forwarded to the Admin for approval. Users must receive notifications regarding the status of their withdrawal.

3.3.10 Leave Review

The system must enable Clients and Freelancers to leave ratings and reviews for each other after project completion. Review submissions must include a numeric rating and optional comments. The system must validate inputs and ensure that reviews can only be submitted by users directly involved in the project. Reviews must appear on user profiles to maintain transparency and trust.

3.3.11 Admin Manage Users

The system must provide Admins with tools to oversee and manage user accounts. Admins must be able to view all registered users, block or suspend accounts, edit user details, and remove accounts when necessary. The system must log all admin actions for security and audit purposes. Admins must only access this module through secure authentication.

3.3.12 Admin Manage Jobs & Disputes

The system must allow Admins to monitor job activities, resolve disputes, and manage reported issues. Admins should be able to review submitted evidence, make decisions, and update dispute statuses. The system must record the outcome and notify both parties. Admins must also have the ability to remove fraudulent job posts or suspend users involved in violations.

3.3.13 Approve Job Proposals

The system must allow Admins to review job proposals before they are publicly listed. Admins must have the ability to approve, reject, or return proposals for revision. The system must validate that each proposal contains all required information. Approved jobs must be published automatically on the platform.

3.3.14 Approve Withdrawals

The system must allow Admins to review and approve withdrawal requests from Freelancers. Admins must verify payment details and available balance before marking the request as approved. The system must process the payment and send confirmation notifications. All withdrawal actions must be logged for audit purposes.

3.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes, constraints, and performance standards that the **TaskHive** must satisfy to ensure a secure, scalable, and efficient experience for all users. Unlike functional requirements, which focus on what the system does, non-functional requirements define *how well* the system performs its functions. These

requirements ensure reliability, usability, maintainability, and performance, supporting the platform's long-term sustainability and user satisfaction.

3.4.1 Performance Requirements

The TaskHive platform must maintain fast and consistent performance across all its core features, including task management, workspace collaboration, activity logging, and AI-powered recommendations. Under normal operational load, the system should respond to user actions—such as creating tasks, updating status, performing searches, or loading dashboards—within **2–3 seconds**.

Backend REST APIs and database operations must be optimized using efficient indexing, pagination, and connection pooling to support multiple concurrent users without degradation. Frequently accessed data, such as user profiles, task lists, and workspace summaries, should implement **server-side caching** to reduce response times. TaskHive must also scale horizontally during peak usage and maintain stable performance during simultaneous interactions from team members.

3.4.2 Security Requirements

TaskHive must follow robust security standards to ensure data protection, access control, and safe communication:

- All communication between client and server must use **HTTPS with TLS encryption**.
- User passwords must be securely hashed using algorithms like **bcrypt** or **argon2**.
- JWT tokens or OAuth 2.0 must be used for session management with proper expiration settings.
- Role-based access control (RBAC) must restrict unauthorized access to workspaces, tasks, and admin features.
- Input validation and sanitization must prevent SQL injection, XSS, CSRF, and other common vulnerabilities.
- Sensitive data (user info, workspace details, task metadata) must not be publicly exposed in API responses or logs.

3.4.3 Usability Requirements

TaskHive must ensure a clean, intuitive, and user-friendly interface:

- The UI should support easy navigation with clear labels, interactive dashboards, and real-time updates.
- Users must be able to create, edit, and assign tasks with minimal clicks.
- The system should provide meaningful error messages, tooltips, and confirmation dialogues.
- Accessibility standards (WCAG 2.1) should be partially supported, including readable fonts, proper contrast, and keyboard navigation.

3.4.4 Reliability Requirements

The platform must be stable and available:

- TaskHive should maintain **99% uptime** under normal conditions.
- In case of errors or system failure, recovery should occur within a minimal time frame without losing user data.
- Automatic retries must handle temporary server or network interruptions.

3.4.5 Scalability Requirements

TaskHive must be designed to scale as the number of users, workspaces, and tasks increases:

- The architecture must allow horizontal scaling of backend services and databases.
- The system should handle increased workload without performance degradation.
- Cloud-based storage and compute resources must auto-scale during peak traffic.

3.4.6 Maintainability Requirements

To ensure long-term sustainability:

- The system must follow clean architecture principles with modular code.
- Documentation must be provided for APIs, database schema, and deployment processes.
- Code should follow naming conventions, version control (Git), and continuous integration practices.
- Developers must be able to add or update features with minimal impact on existing modules.

3.4.7 Availability Requirements

- The system must remain accessible 24/7 except during scheduled maintenance.
- Maintenance windows should be communicated to users in advance.

3.4.8 Compatibility Requirements

- TaskHive must be compatible with modern web browsers (Chrome, Edge, Firefox, Safari).
- The platform should be responsive and functional on desktops, tablets, and mobile devices.
- APIs must follow REST standards to ensure easy integration with third-party apps.

Chapter 4

DESIGN AND ARCHITECTURE

4 Design and Architecture

In the Design and Architecture phase for TaskHive, we define the structural blueprint that governs how the platform's components interact to deliver a reliable, secure, and user-friendly freelancing marketplace. This phase covers both the user-facing aspects (responsive, accessible UI for clients, freelancers, and admins) and the server-side architecture (scalable APIs, real-time services, persistence, and integrations). The goal is to produce a modular, maintainable system that supports rapid feature evolution (bids, messaging, milestones, payments) while ensuring performance, security, and testability.

4.1 System Architecture

TaskHive adopts a **three-tier architecture** that separates concerns into Presentation, Application (Logic), and Data layers. This separation improves modularity, enables independent scaling of layers, and simplifies testing and maintenance.

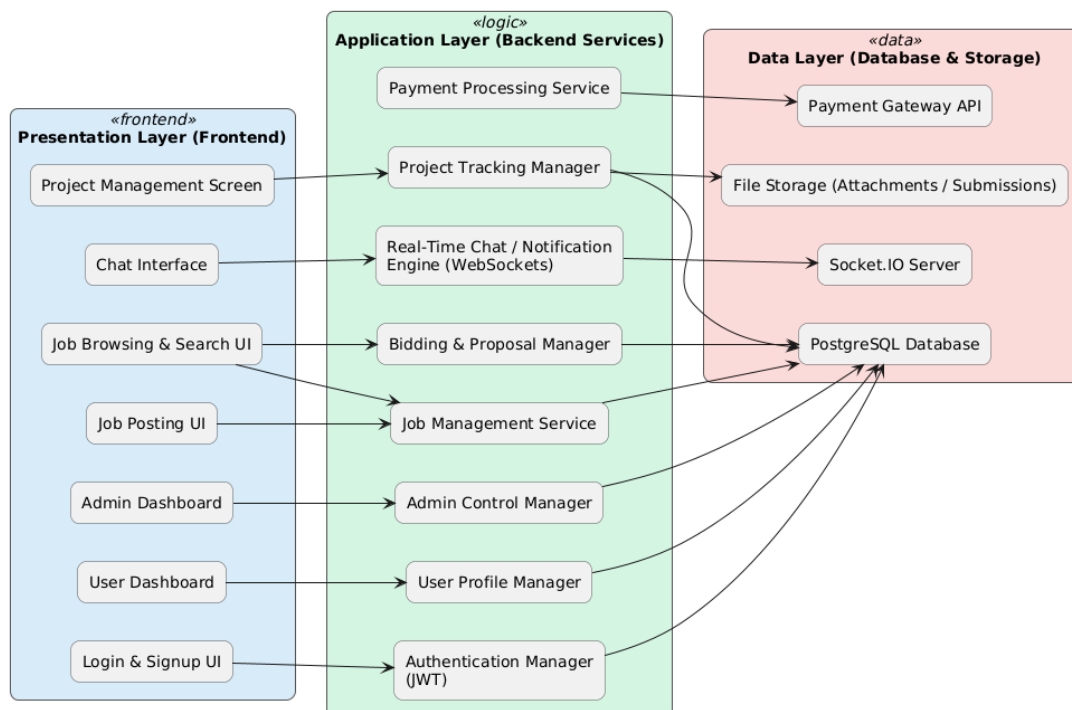


Figure 4.1: TaskHive System architecture

Explanation

1. Presentation Layer (Frontend)

- **Purpose:** Render UIs and handle user interactions for Clients, Freelancers, and Admins.
- **Responsibilities:** authentication UI, job posting forms, job browsing/search, bid submission, real-time chat UI, project/milestone dashboard, payment flows, notification panel, and admin consoles.

2. Application Layer

- **Purpose:** Host core business rules, REST APIs, and real-time communication logic.
- **Responsibilities:** authentication & authorization (JWT), user/profile management, job lifecycle (create → bid → accept → milestone → complete), bidding & proposal management, project tracking, notifications and presence, payment orchestration (gateway integration and escrow-like flow), admin operations (moderation, dispute handling, analytics), and validation/security.
- **Real-time services:** Socket.IO or similar for chat, typing indicators, read receipts, and event notifications.

3. Data Layer

- **Purpose:** Durable storage and third-party integrations.
- **Responsibilities:** store users, profiles, jobs, bids, chat history, milestones, transactions, audit logs, and analytics. Integrate with payment gateway APIs, file storage for attachments/submissions, and optionally external identity or KYC services.

4.2 Data Representation

The Data Representation section defines how TaskHive stores, structures, and manages information within the platform. It outlines the major data entities, their attributes, and the

relationships between them. A well-designed data model ensures consistency, reduces redundancy, and provides a robust foundation for critical operations such as job posting, bidding, payments, project tracking, and communication.

The TaskHive database follows a **relational schema**, ensuring transactional consistency for payment workflows, proposal handling, and dispute resolution. Each entity is uniquely identified and is linked using foreign keys to maintain strong referential integrity.

4.2.1 Key Entities and Relationships

User – Stores authentication and profile details for clients, freelancers, and admins. Includes name, email, role, skills, and account status.

Job – Represents job postings created by clients. Contains title, budget, description, category, and deadline.

Bid / Proposal – Contains freelancer offers for a job, including proposed amount, delivery time, and proposal details.

Project – Created when a client accepts a freelancer’s bid. Tracks deadlines, deliverables, progress, and collaboration details.

Payment – Manages financial transactions between clients and freelancers for completed work or milestones.

Withdrawal – Handles freelancer requests to transfer earnings to their bank or wallet.

Message – Stores real-time chat communication between clients and freelancers within a project.

Review – Contains feedback, ratings, and comments exchanged after a project is completed.

Notification – Stores system alerts for job updates, proposals, messages, payments, and withdrawals.

Relationships:

1. One **User** can post multiple **Jobs** (Client).
2. One **User** can submit multiple **Bids** (Freelancer).
3. Each **Job** can have multiple **Bids**.
4. One approved **Bid** creates a **Project**.
5. A **Project** can have multiple **Messages**, **Payments**, and **Reviews**.
6. A **Freelancer** can submit multiple **Withdrawal** requests.
7. Each **User** can receive multiple **Notifications**.
8. Admins manage **Users**, **Jobs**, **Disputes**, and **Withdrawals**.

4.2.2 ER Diagram

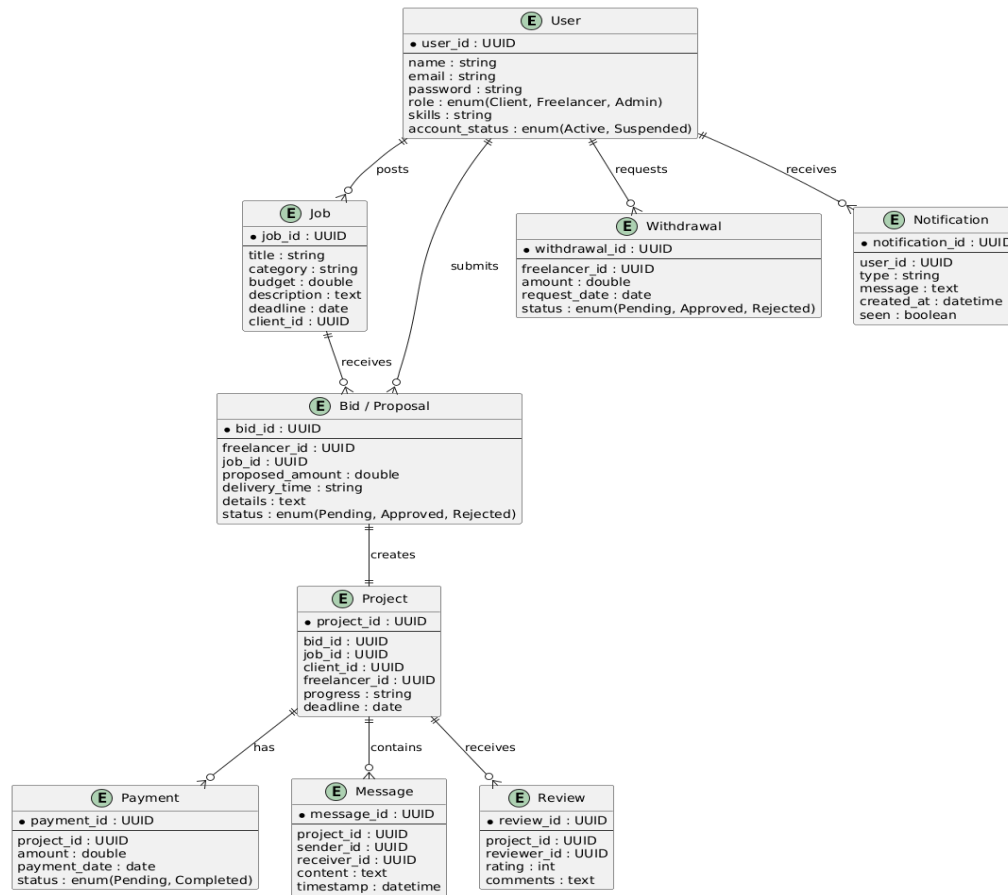


Figure 4.2: TaskHive ER Diagram

Explanation

The ER Diagram of TaskHive represents all core entities involved in the platform, including Users, Jobs, Bids, Projects, Payments, Withdrawals, Messages, Reviews, and Notifications. It visually illustrates how clients, freelancers, and admins interact with the system through job posting, bidding, collaboration, and financial workflows. The relationships highlight the flow from job creation to bid approval, project execution, communication, payment processing, and final feedback. This structured model ensures efficient data organization, consistency, and smooth interaction between all system modules.

4.3 Process Flow/Representation

The Process Flow Representation illustrates how different operations within TaskHive are executed from start to finish. It outlines the sequential flow of activities across core modules such as job posting, bidding, project management, communication, and payments. These flows help visualize how clients, freelancers, and admins interact with the system, ensuring clarity in system behavior and supporting efficient design, development, and testing.

4.3.1 Activity Diagrams

Activity Diagrams depict the step-by-step workflows of major TaskHive processes such as user authentication, job posting, bidding, project execution, and withdrawal approval. Each diagram captures key actions, decision points, parallel activities, and system responses that occur during user interactions. These diagrams help in understanding the operational logic of the system, identifying dependencies, and ensuring smooth coordination among different system modules.

4.3.1.3 User Login and Registration Activity Diagram

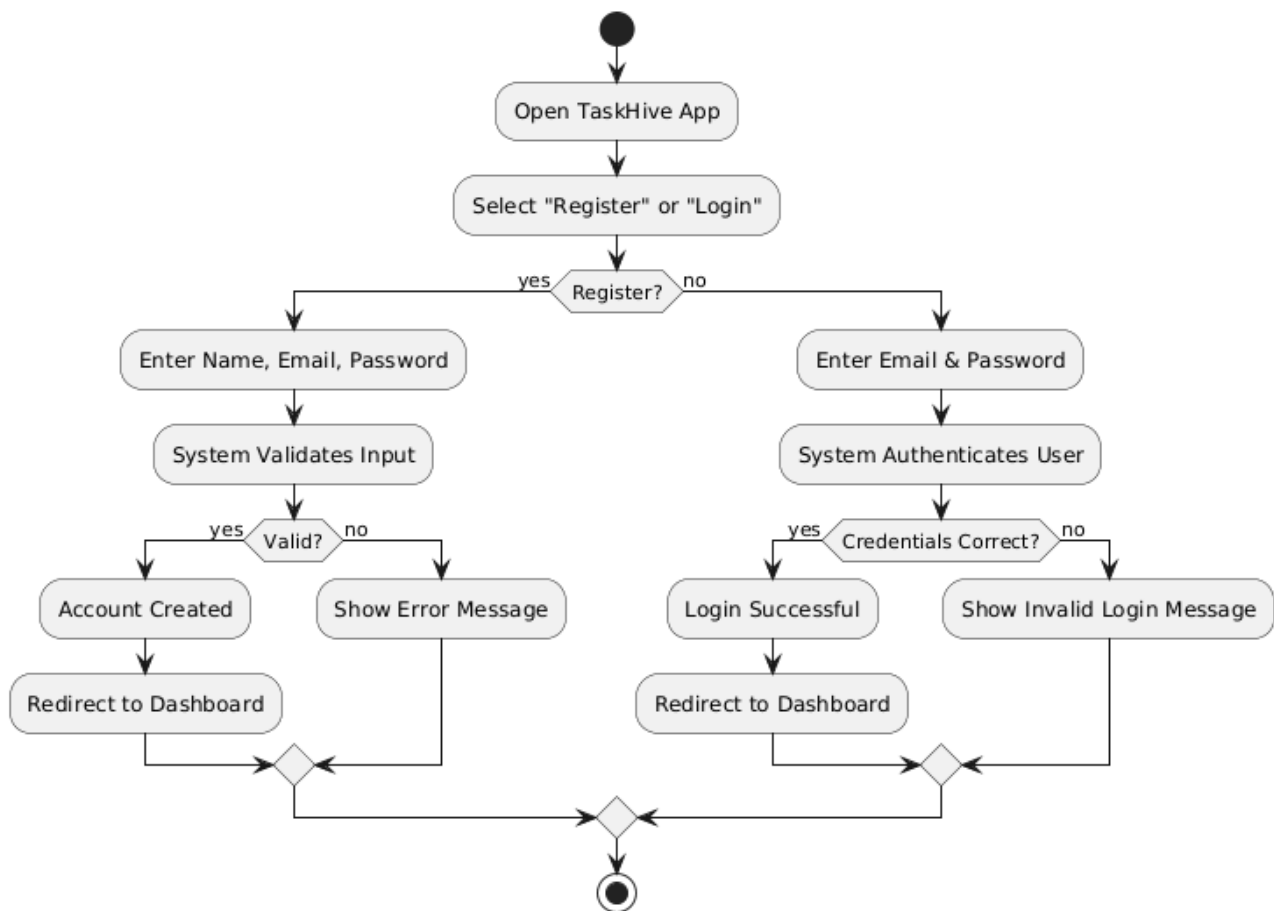


Figure 4.3: User Login and Registration Activity Diagram

Figure 4.3 illustrates activity diagram represents the steps a user follows to register or log into TaskHive. It shows validation of input credentials and system responses for successful or failed authentication. The flow ensures secure access for clients, freelancers, and admins. Errors such as invalid credentials or duplicate accounts are also handled.

4.3.1.4 Job Posting (Client)Activity Diagram

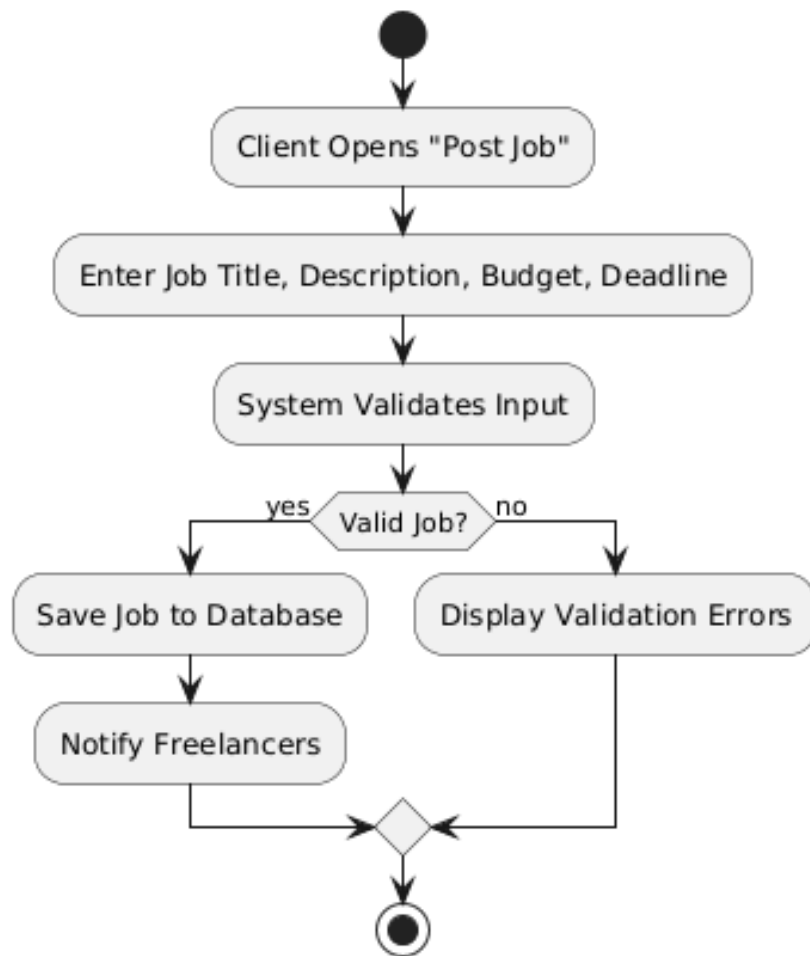


Figure 4.4: *Job Posting (Client) Activity Diagram*

Figure 4.4 illustrates the process a client follows to create a new job posting. It includes entering job details, validating input fields, and saving the job into the system. Once posted, notifications are sent to freelancers, helping them discover relevant opportunities. Invalid entries prompt corrective feedback.

4.3.1.5 Bidding on Jobs (Freelancer) Activity Diagram

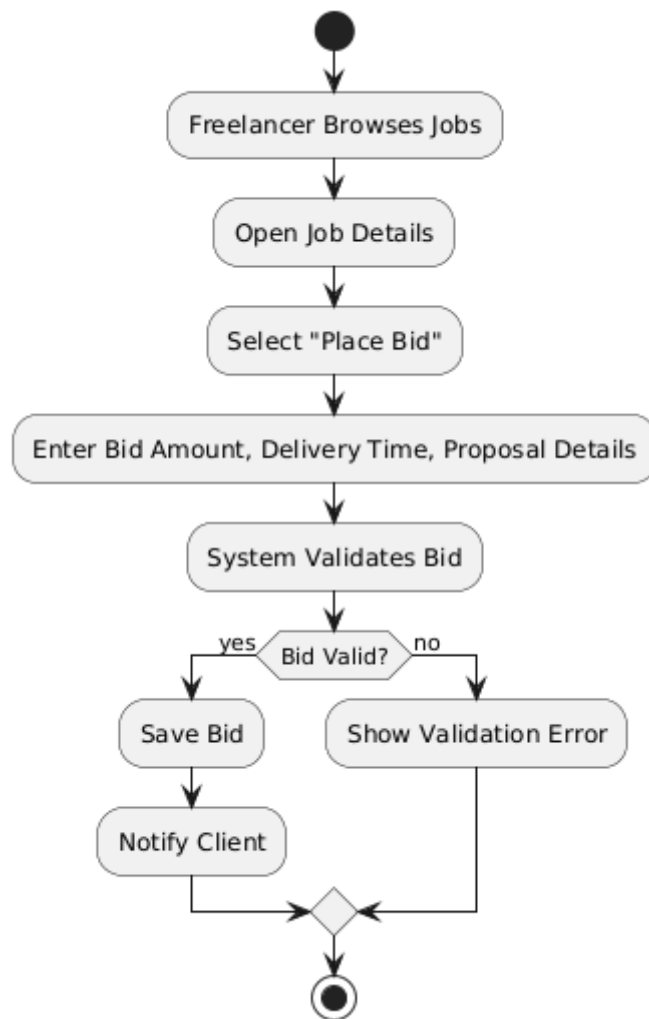


Figure 4.5: *Bidding on Jobs (Freelancer) Activity Diagram*

Figure 4.5 This activity diagram shows how freelancers explore available jobs, review details, and submit their proposals. The system validates the bid and notifies the client upon successful submission. Any incorrect or incomplete bid information results in validation errors. This workflow supports fair competition and efficient proposal management.

4.3.1.6 Project Execution Workflow Activity Diagram

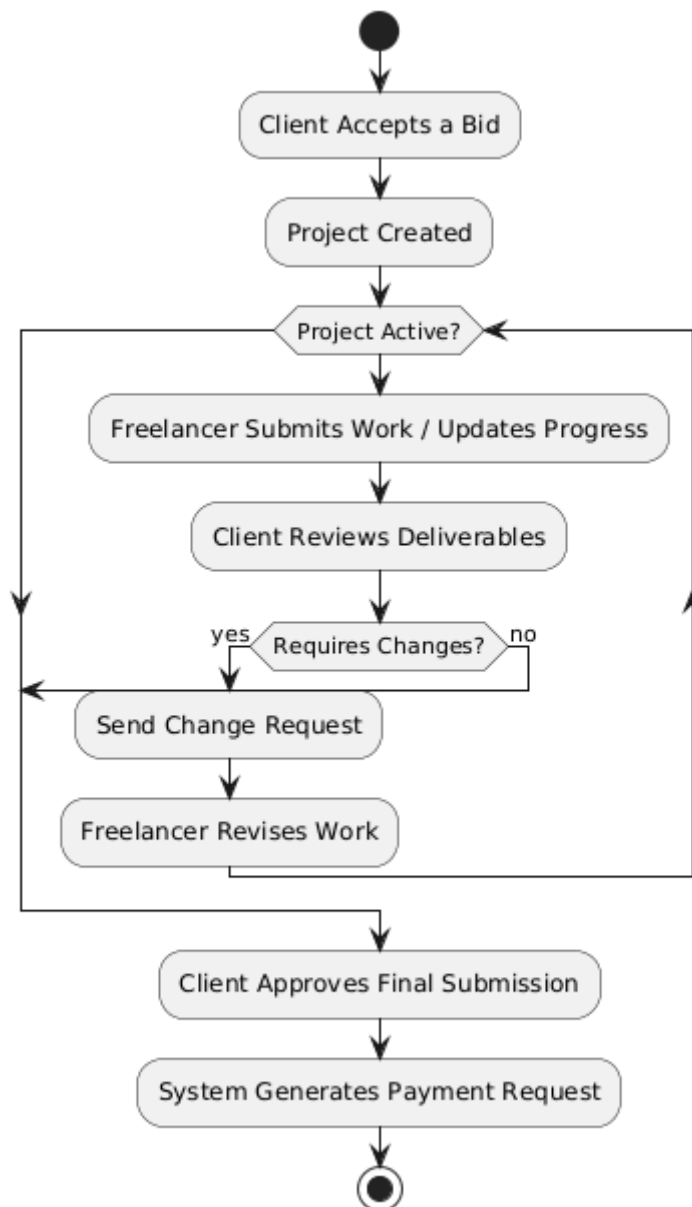


Figure 4.6: Project Execution Workflow Activity Diagram

Figure 4.6 This diagram outlines the collaboration cycle after a client accepts a bid and a project begins. It covers progress updates, work submissions, change requests, and final approvals. The loop ensures iterative refinement until the client is satisfied. The process ends with payment initiation once the final deliverables are approved.

4.3.1.7 Withdrawal Request Process Activity Diagram

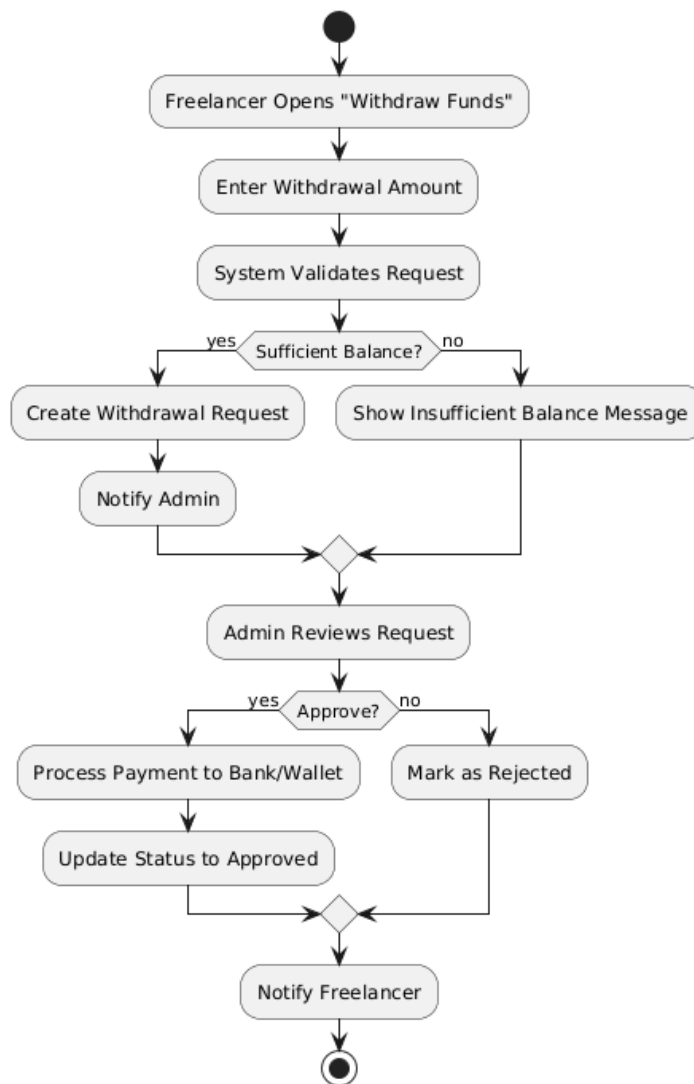


Figure 4.7: Withdrawal Request Process Activity Diagram

Figure 4.7 This activity diagram explains how freelancers request withdrawal of their earnings and how admins review these requests. It includes checking available balance, creating requests, and processing approvals or rejections. Notifications keep freelancers informed of each update. The flow ensures secure and transparent financial transactions.

4.4 Design Models

4.4.1 Sequence Diagram

Sequence Diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration. Sequence Diagrams are time focus, and they show the order of the interaction visually by using the vertical axis of the diagram to represent time what messages are sent.

4.4.1.1 Sequence Diagram for User Registration and Login

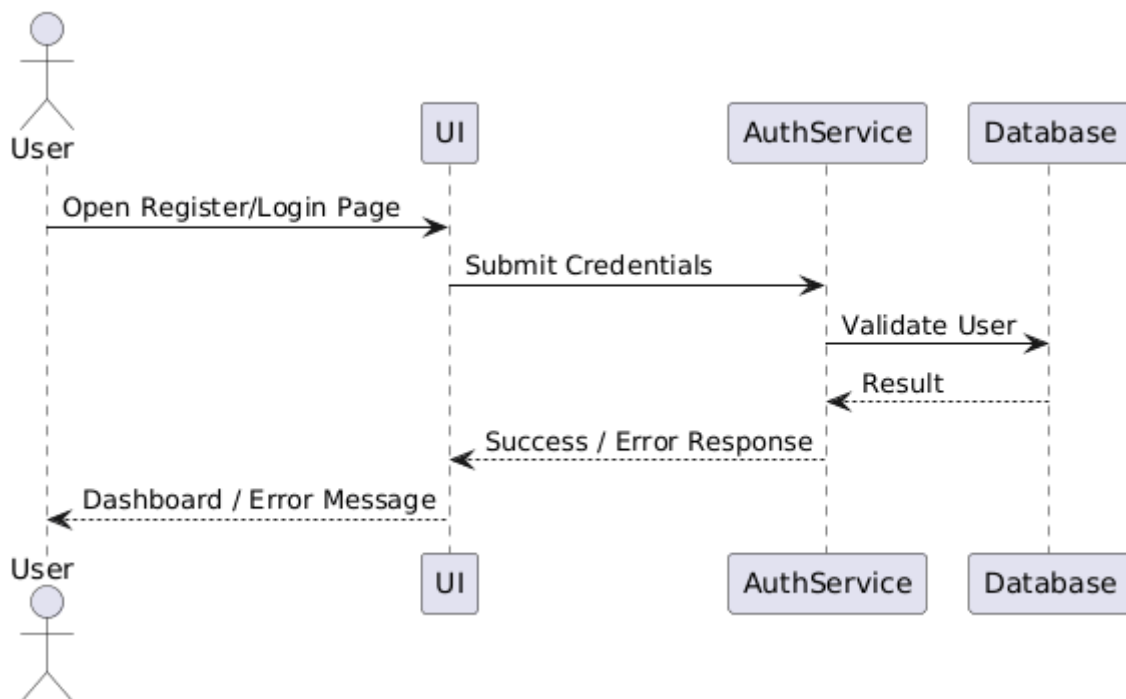


Figure 4.8: Sequence Diagram for User Registration and Login

Figure 4.9 This sequence diagram illustrates how users initiate registration or login, submit credentials, and receive authentication responses. The system validates email/password, checks database records, and returns success or failure. Secure session tokens are issued upon successful login, ensuring protected access to the platform.

4.4.1.2 Sequence Diagram for Job Posting (Client)

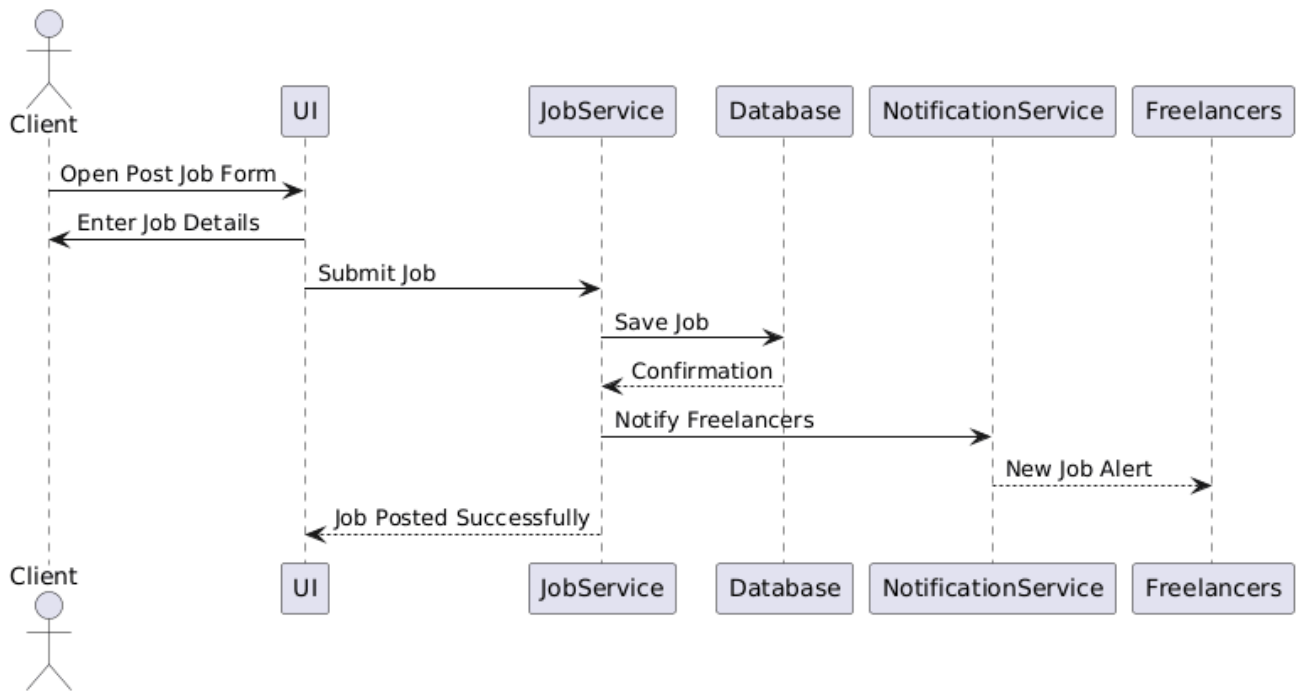


Figure 4.9: Job Posting (Client) Sequence Diagram

Figure 4.9 This diagram shows how a client creates and submits a job posting. The system validates the job details, saves them to the database, and notifies relevant freelancers. Successful posting results in the job appearing in the job marketplace for freelancers to browse and bid on.

4.4.1.3 Sequence Diagram for Bid Submission (Freelancer)

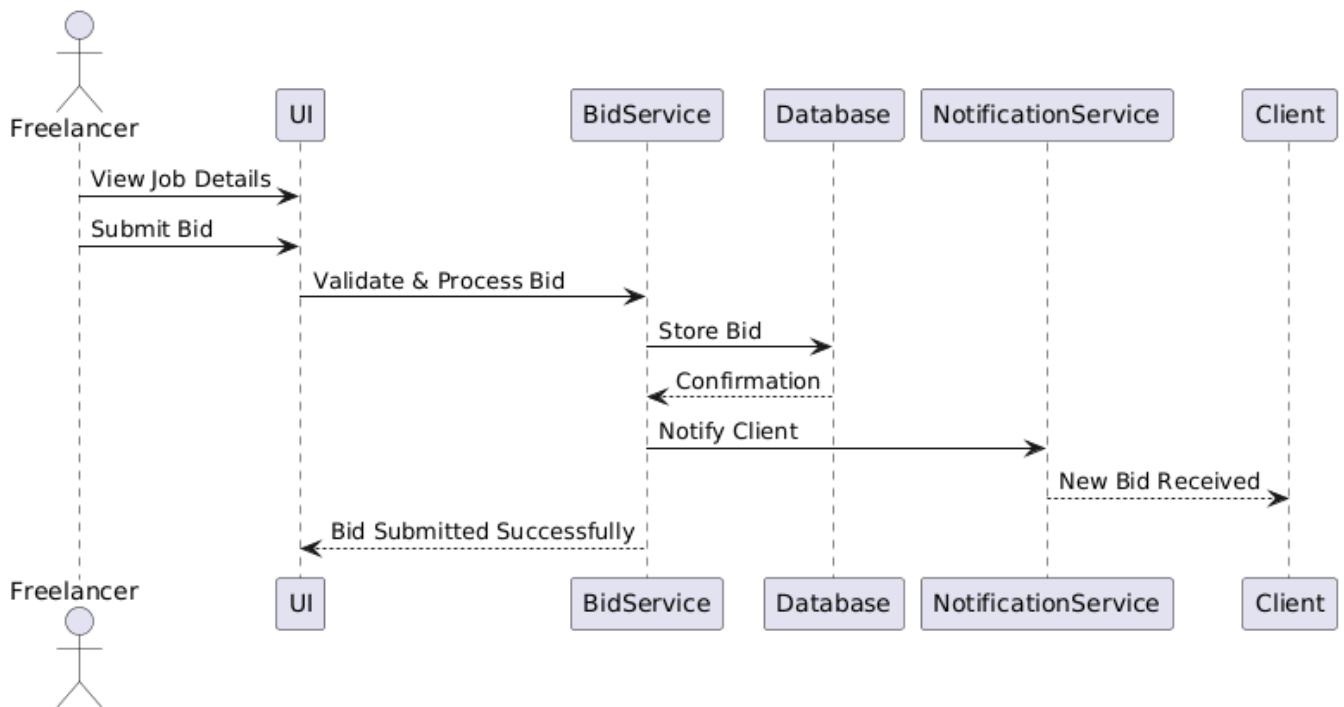


Figure 4.10: Bid Submission (Freelancer) Sequence Diagram

Figure 4.10 This sequence diagram demonstrates how freelancers browse jobs and submit a bid. The system checks whether the bid details are valid and stores the proposal. Once submitted, the client receives a notification informing them of a new proposal.

4.4.1.4 Sequence Diagram for Project Execution Workflow

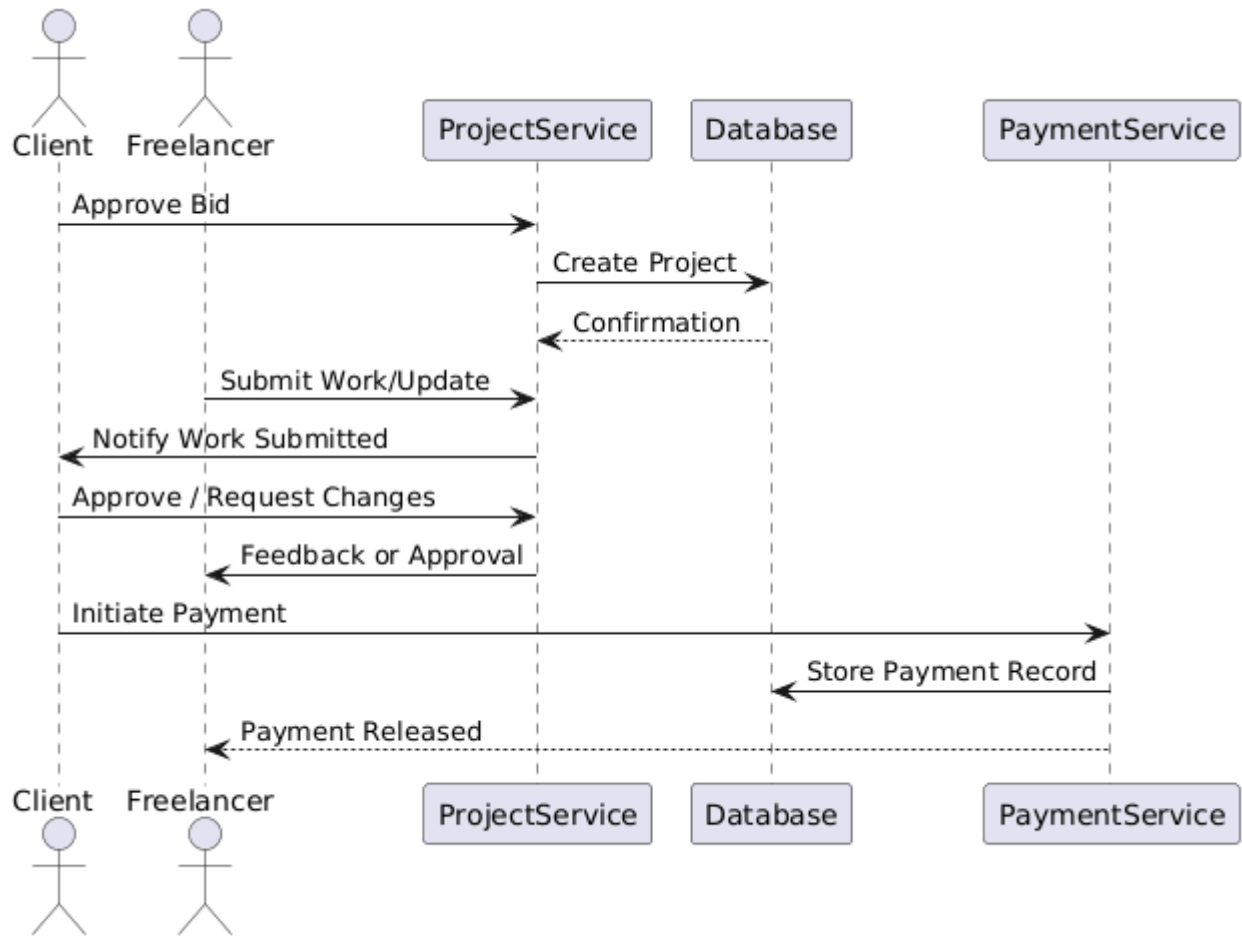


Figure 4.11: Project Execution Workflow Sequence Diagram

Figure 4.11 This diagram explains how collaboration happens once a bid is approved and the project starts. Freelancers send updates or deliverables, clients review them, and the process continues until approval. Upon completion, the system triggers payment processing for the freelancer.

4.4.1.5 Sequence Diagram for Withdrawal Request Process

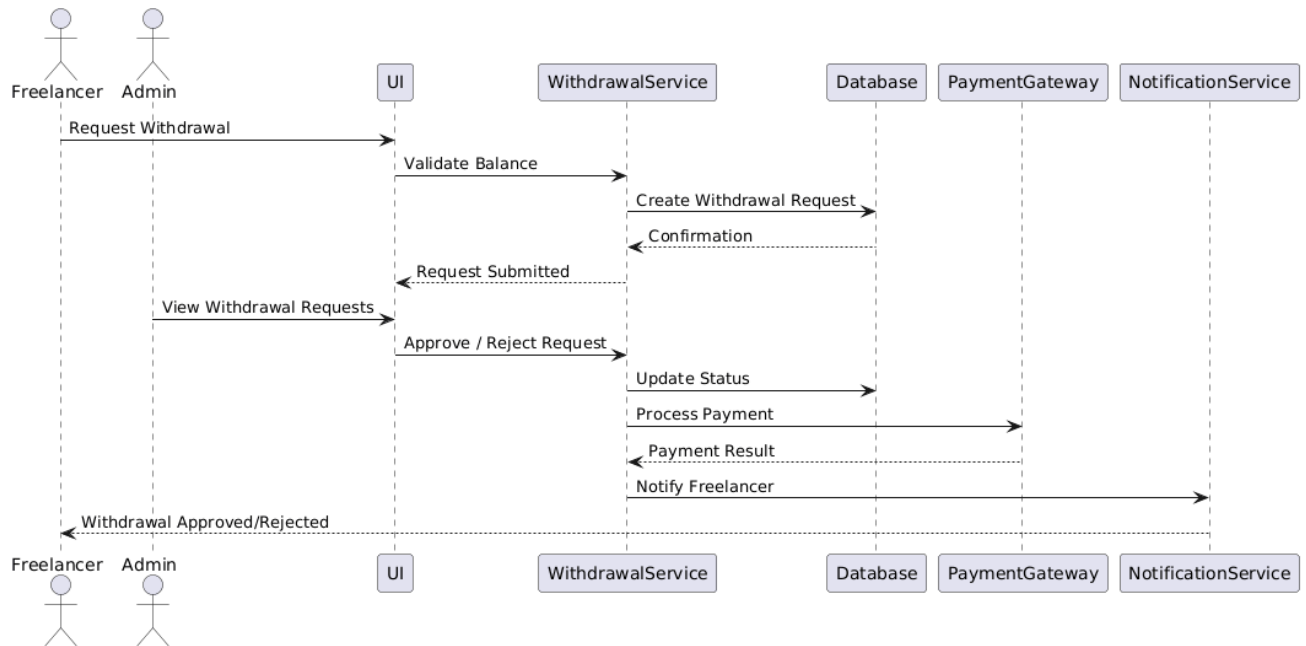


Figure 4.12: Withdrawal Request Process Sequence Diagram

Figure 4.12 This diagram illustrates how freelancers request withdrawals and how admins approve or reject them. The system verifies balance availability, logs the request, and waits for admin action. After approval, the payment gateway processes the withdrawal, and the freelancer is notified.

4.4.2 Class Diagram

A class diagram is a static structure diagram in UML that represents the structure of a system by showing its classes, attributes, methods, and relationships. It is widely used in object-oriented systems to visualize, document, and construct executable code. Class diagrams describe the static view of an application and are crucial for modeling system functionalities and collaborations.

4.4.2.1 Class Diagram

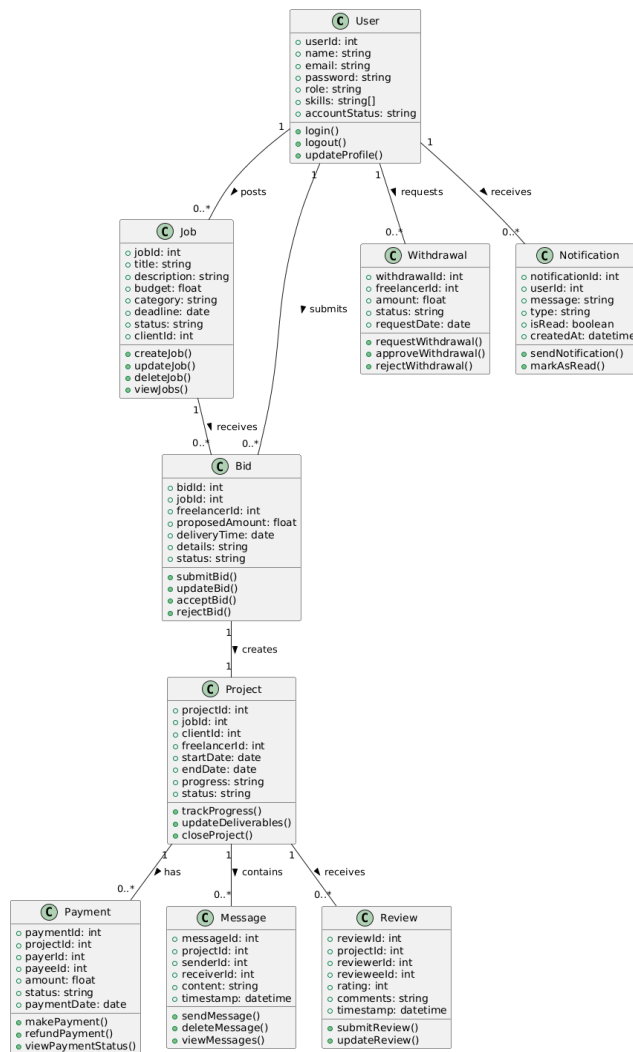


Figure 4.13: Class Diagram

Figure 4.13: Class Diagram of TaskHive Platform

The class diagram illustrates the core entities and their relationships in the **TaskHive platform**:

1. User Class:

Represents all users of the platform, including clients, freelancers, and admins. Attributes include `userId`, `name`, `email`, `password`, `role`, `skills`, and `accountStatus`. Methods allow users to `login()`, `logout()`, and `updateProfile()`.

Clients post jobs, freelancers submit bids, and admins manage users, jobs, disputes, and withdrawals.

2. **Job Class:**

Represents job postings created by clients.

Attributes include jobId, title, description, budget, category, deadline, status, and clientId.

Methods allow users to createJob(), updateJob(), deleteJob(), and viewJobs().

Jobs can receive multiple bids from freelancers.

3. **Bid / Proposal Class:**

Represents freelancer offers for a job.

Attributes include bidId, jobId, freelancerId, proposedAmount, deliveryTime, details, and status.

Methods include submitBid(), updateBid(), acceptBid(), and rejectBid().

Accepted bids create a new project.

.

4. **Project Class:**

Represents a project created when a client accepts a bid.

Attributes include projectId, jobId, clientId, freelancerId, startDate, endDate, progress, and status.

Methods include trackProgress(), updateDeliverables(), and closeProject().

A project can have multiple messages, payments, and reviews associated with it.

5. **Payment Class:**

Manages financial transactions between clients and freelancers.

Attributes include paymentId, projectId, payerId, payeeId, amount, status, and paymentDate. Methods include makePayment(), refundPayment(), and viewPaymentStatus().

6. **Withdrawal Class:**

Handles freelancer requests to transfer earnings.

Attributes include withdrawalId, freelancerId, amount, status, and requestDate.

Methods include requestWithdrawal(), approveWithdrawal(), and rejectWithdrawal().

7. **Message Class:**

Stores real-time chat communication between clients and freelancers within a project.

Attributes include messageId, projectId, senderId, receiverId, content, and timestamp.

Methods include sendMessage(), deleteMessage(), and viewMessages().

8. **Review Class:**

Stores feedback and ratings exchanged after project completion.

Attributes include reviewId, projectId, reviewerId, revieweeId, rating, comments, and timestamp.

Methods include submitReview() and updateReview().

9. **Notification Class:**

Stores system alerts for job updates, proposals, messages, payments, and withdrawals.

Attributes include notificationId, userId, message, type, isRead, and createdAt.

Methods include sendNotification() and markAsRead().

Relationships:

- **User → Job:** A client can post multiple jobs.
- **User → Bid:** A freelancer can submit multiple bids.
- **Job → Bid:** Each job can receive multiple bids.
- **Bid → Project:** One accepted bid creates a single project.
- **Project → Message:** A project can have multiple messages.
- **Project → Payment:** A project can have multiple payments or milestones.
- **Project → Review:** A project can have multiple reviews.
- **User → Withdrawal:** A freelancer can submit multiple withdrawal requests.
- **User → Notification:** Users can receive multiple notifications.

Chapter 5

IMPLEMENTATION

5 Implementation

In the implementation phase of TaskHive, we have developed algorithms to power the core functionalities of the platform, enabling secure authentication, efficient job posting, seamless bidding, real-time messaging, and smooth financial transactions between clients and freelancers. These algorithms ensure reliable operations and a user-friendly experience for all stakeholders.

5.1 Algorithm

5.1.1 User authentication

Function: authenticateUser (email, password)

Input: email (string), password (string)

Output: user (object) or error (string)

Description:

Handles secure authentication for Clients, Freelancers, and Admins using **bcrypt** and **JWT** tokens.

Pseudocode:

1. Retrieve user record from the database using email.
2. If user exists:
 - a. Compare entered password with hashed password.
 - b. If match: generate JWT token and return user object.
 - c. If not: return "Incorrect password."
3. If no user found: return "User not found."

5.1.2 Job Posting

Function: createJob(clientId, jobDetails)

Input: clientId (string), jobDetails (object: title, description, budget, category, deadline)

Output: job (object) or error (string)

Description:

Allows clients to post jobs that freelancers can bid on.

Pseudocode:

1. Validate jobDetails for required fields.
2. Generate unique jobId.
3. Associate jobId with clientId.
4. Save job in database.
5. Return job object.

5.1.3 Bid Submission

Function: submitBid(freelancerId, jobId, bidDetails)

Input: freelancerId (string), jobId (string), bidDetails (object: proposedAmount, deliveryTime, details)

Output: bid (object) or error (string)

Description:

Allows freelancers to submit bids for jobs. Each freelancer can submit one active bid per job.

Pseudocode:

1. Check if freelancer has an active bid for jobId.
2. If not, generate bidId.
3. Associate bid with freelancerId and jobId.
4. Save bid in database.
5. Return bid object.

5.1.4 Project Creation

Function: createProject(acceptedBidId)

Input: acceptedBidId (string)

Output: project (object) or error (string)

Description:

Creates a project when a client accepts a bid, linking client, freelancer, and job.

Pseudocode:

1. Retrieve bid and job from database using acceptedBidId.
2. Generate projectId.
3. Initialize project attributes: startDate, endDate, progress, status.
4. Save project in database.
5. Return project object.

5.1.5 Payment Processing

Function: processPayment(payerId, payeeId, projectId, amount)

Input: payerId, payeeId, projectId, amount

Output: payment (object) or error (string)

Description:

Handles secure payments for completed jobs or milestones.

Pseudocode:

1. Validate payment amount and project status.
2. Process payment via gateway API.
3. On success, save payment in database.
4. Update project payment status.
5. Return payment confirmation.

5.1.6 Withdrawal Request

Function: requestWithdrawal(freelancerId, amount)

Input: freelancerId (string), amount (float)

Output: withdrawal (object) or error (string)

Description:

Handles freelancer requests to transfer earnings to bank or wallet.

Pseudocode:

1. Validate freelancer balance.
2. Generate withdrawalId.
3. Save withdrawal request in database with status pending.
4. Admin approves or rejects request.
5. Update withdrawal status and freelancer balance.
6. Return withdrawal object.

5.1.7 Notifications

Function: sendNotification(userId, message, type)

Input: userId (string), message (string), type (string: job, bid, project, payment, withdrawal)

Output: notification (object)

Description:

Sends system alerts for jobs, bids, messages, payments, and withdrawals.

Pseudocode:

1. Generate notificationId.
2. Associate notification with userId and type.
3. Save message and timestamp in database.
4. Mark as unread initially.

5. Return notification object

These are simplified pseudocode representations of the algorithms for each core component of the TaskHive Platform. The actual implementation may include additional error handling, input validation, and integration with frontend components. It also involves real-time interactions between the user interface, backend APIs, and the PostgreSQL database.

5.2 External APIs

Table 5.2.1: External APIs

Library / API	Purpose
Axios	Handles REST API requests between frontend and backend.
Express.js	Framework for building RESTful backend APIs and routing.
PostgreSQL	Relational database for storing users, jobs, bids, projects, payments, reviews, and notifications.
Prisma	ORM/ODM library to model and interact with PostgreSQL data.
WebSockets/ Socket.IO	Enables real-time messaging and notifications between clients and freelancers.
JWT (JSON Web Token)	Used for secure authentication, session management, and role-based access control.
Bcrypt.js	Password hashing and verification for secure login.
Axios	Handles REST API requests between frontend and backend.

Table 5.2.1: External APIs lists that TaskHive leverages a modern full-stack architecture combining React + Tailwind CSS on the frontend and Node.js + Express on the backend. PostgreSQL serves as the relational database, with Prisma providing ORM capabilities. Real-time collaboration is enabled using WebSockets/Socket.IO, while JWT and Bcrypt.js secure authentication. Together, these technologies ensure a secure, responsive, and feature-rich freelance platform.

5.3 User Interface

The StayBnB Real Estate Platform is designed with a user-centric approach, ensuring that individuals can easily navigate and utilize its features without technical expertise. The interface is modern, responsive, and intuitive, allowing both users and admins to manage their tasks efficiently. Each section of the platform is organized into clear workflows that mirror real-world property rental and management processes. Below is a detailed explanation of each module and its interaction flow.

5.3.1 Sign-in Screen

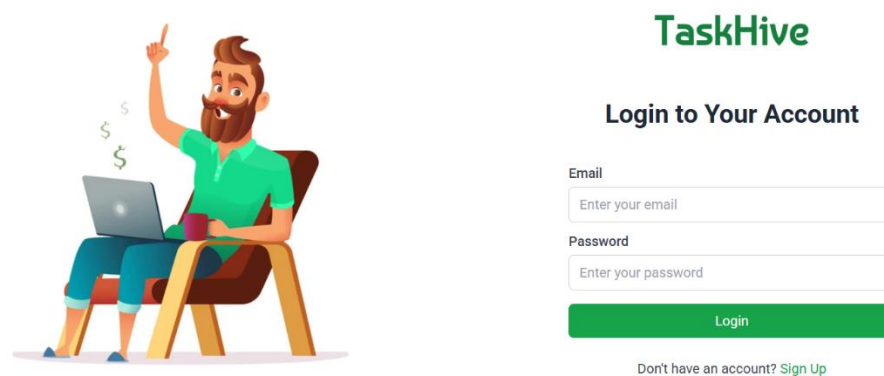


Figure 5.1: Sign-in Screen

5.3.2 Sign-up Screen



Client

TaskHive

Create New Account

First Name

Last Name

User Name

Email

Password

Confirm Password

Register

Already have an account? [Log in](#)

Figure 5.2: Sign-up Screen

Figure 5.1 & 5.2: The Sign-in and Sign-up Screen of the TaskHive platform provides a simple and secure entry point for users and administrators. It includes input fields for the user’s email address and password, along with quick access options to either sign in or create a new account. The interface ensures ease of use through a minimal and modern design, offering clear navigation and visual cues. When a user enters valid credentials and clicks “Sign in”, the system validates their data via the backend API using JWT-based authentication. Upon successful login, users are redirected to the main dashboard.

5.3.3 Home Screen

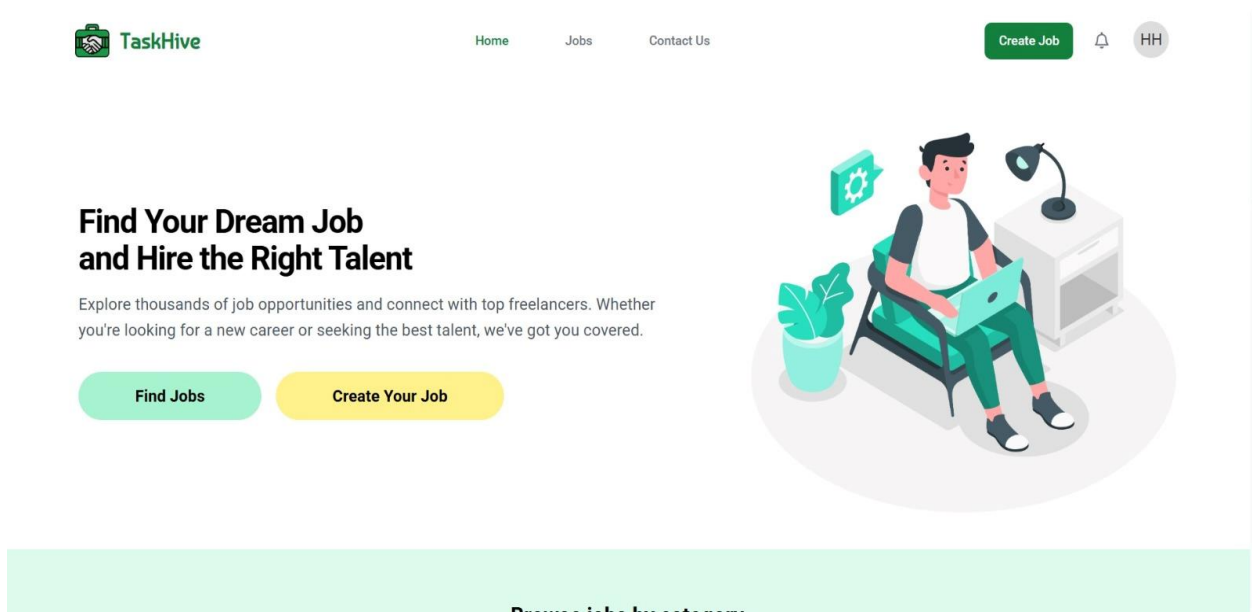


Figure 5.3: Home Screen

Figure 5.3: The TaskHive Homepage acts as the primary gateway for job discovery and talent hiring, featuring a clean and modern layout. It showcases the value proposition: “Find Your Dream Job and Hire the Right Talent,” guiding both job seekers and employers. The homepage includes clear action buttons—**Find Jobs** and **Create Your Job**—along with a welcoming illustration that enhances user engagement. The interface emphasizes simplicity and accessibility, allowing users to easily explore opportunities or post new job listings.

5.3.4 Freelancer Profile Screen

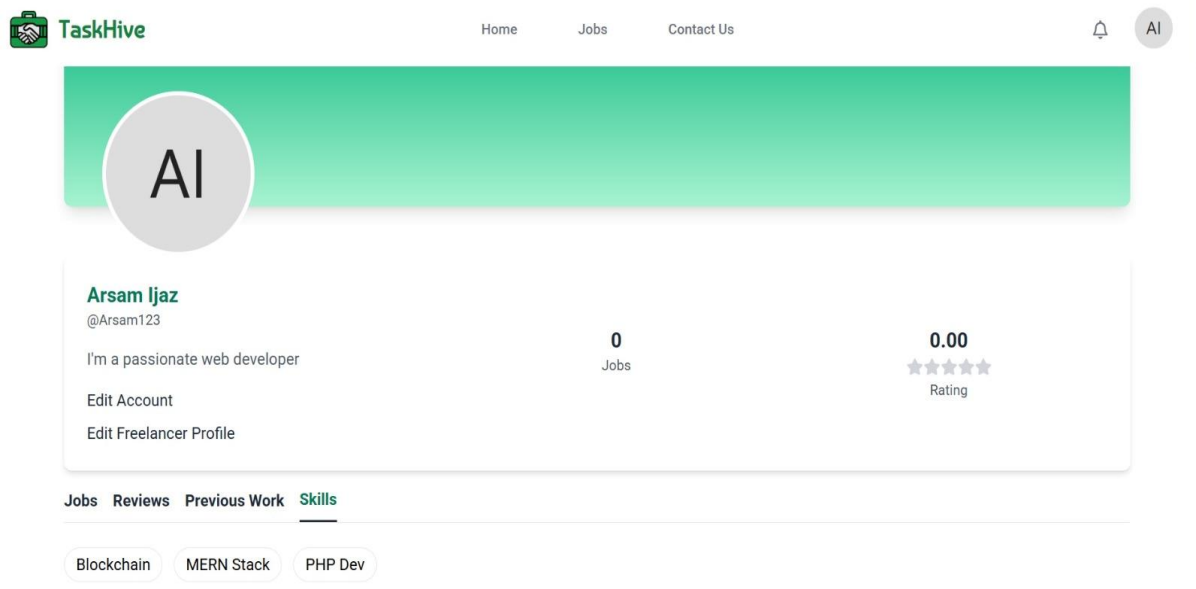


Figure 5.4: Freelancer Profile Screen

Figure 5.4: The Freelancer Profile Page provides a personalized dashboard for professionals on TaskHive, showcasing their identity, bio, and key performance metrics. The page displays the freelancer's name, username, and profile description, along with stats such as total jobs completed and overall rating. It includes quick-access options to edit account details and update the freelancer profile. The lower section offers organized tabs—Jobs, Reviews, Previous Work, and Skills—allowing users to highlight their expertise, with skill tags visually presented for easy recognition.

5.3.5 Manage Job

The screenshot displays the 'Manage Job' interface on the TaskHive platform. At the top, there is a navigation bar with 'Home', 'Jobs', and 'Contact Us' links, along with a notification bell and a user profile icon labeled 'AI'. The main content area is divided into four sections:

- Job Details:** A table showing job information.

Job Title	Job Category
Web Dev	Web Development
Job Description	Status
I want an experienced Web developer for my agency.	In Progress
- Proposal Details:** A table showing proposal information.

Freelancer Name	Freelancer Username	Profile
Arsam Ijaz	Arsam123	View Profile
Total Price	Duration	Submission Date
250\$	5 days	Wed Dec 10 2025
- Messages:** A chat interface showing a message from 'HH' with the text 'yes' and a timestamp of '12/10/2025, 12:43:01 AM'. Below the message is a text input field labeled 'Type a message' and buttons for 'Send' and 'Attach'.
- Milestones:** A section displaying task progress, pricing, and completion status. It shows 'Price: \$150.00', 'Duration: 2 days', and 'Backend API Development' with a 'Pending' status.

Figure 5.5: Manage Job

Figure 5.5: The Manage Job Page provides a comprehensive overview of an ongoing project, combining job details, proposal information, communication, and milestone tracking in a single interface. The top section highlights key information such as the job title, category, current status, freelancer details, pricing, and submission date. Below, an integrated messaging panel enables real-time communication between client and freelancer, ensuring smooth collaboration. On the right, the milestones section displays task progress, pricing, and completion status, allowing both parties to monitor deliverables throughout the project lifecycle.

5.3.6 Admin Dashboard Screen

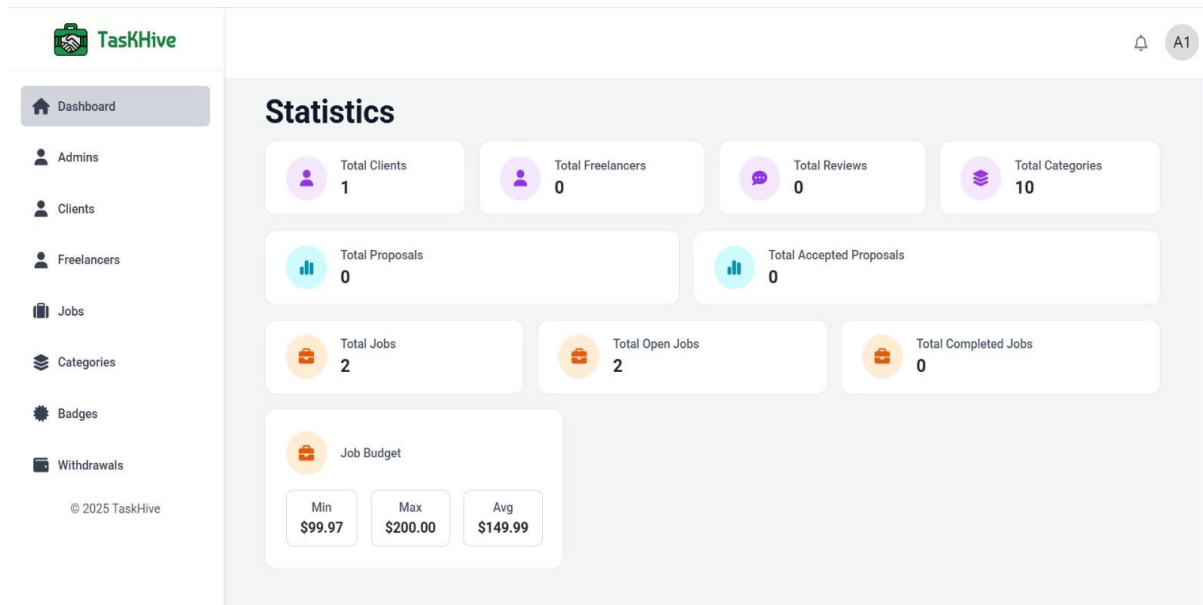


Figure 5.6: Admin Dashboard Screen

Figure 5.6: The Admin Dashboard provides a centralized interface for managing the entire TaskHive platform, offering a clear snapshot of system-wide activity. It displays key metrics including total clients, freelancers, reviews, categories, proposals, and job statistics. The dashboard highlights the number of open, completed, and overall jobs, along with detailed budget insights such as minimum, maximum, and average job values. The sidebar enables quick navigation to admin management, client and freelancer listings, job controls, categories, badges, and withdrawal requests—ensuring efficient administrative oversight and platform monitoring.

5.3.7 Payment

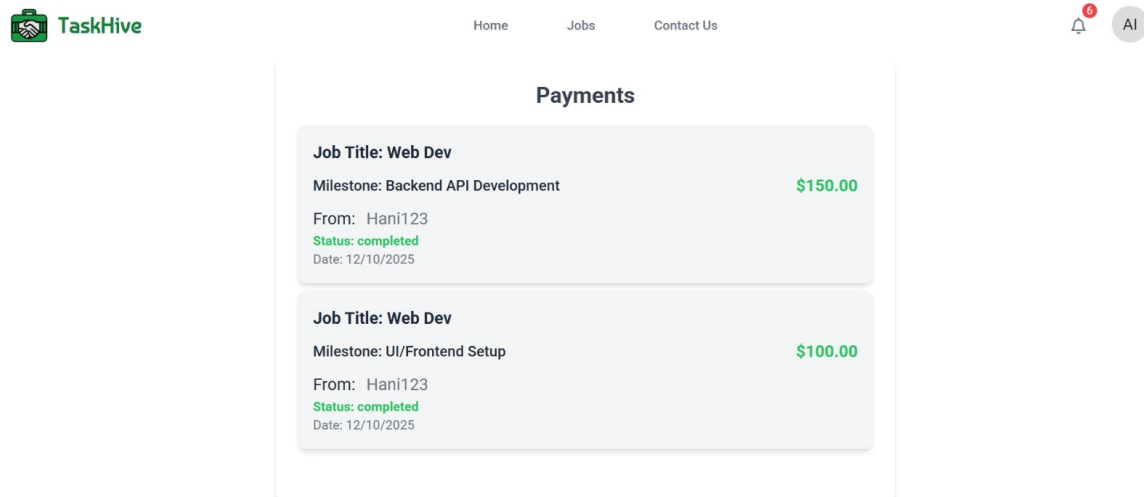


Figure 5.7: Payment Screen

Figure 5.7: The Payments page provides freelancers with a clear overview of all completed milestone payouts. Each payment card displays the job title, specific milestone completed, client username, payment status, and the date of transaction. The amount received is highlighted in green on the right for quick visibility. This streamlined layout helps users easily track their earnings and verify completed work, ensuring full transparency in the payment workflow across the TaskHive platform.

Chapter 6

TESTING AND EVALUATION

6 Testing and Evaluation

Testing is a crucial step in ensuring that the **TaskHive** platform functions correctly, meets user expectations, and provides a smooth freelance ecosystem for clients and freelancers. Testing is carried out at multiple stages of development to detect and fix errors early. Systematic testing helps assess performance, reliability, security, and usability. Following standard software engineering practices, TaskHive treats testing as a dedicated phase after implementation to objectively evaluate the software. Structured testing ensures higher software quality, better user experience, and elimination of major issues before deployment.

6.1 Manual Testing

Manual testing was performed to identify interface, navigation, and functional issues by directly interacting with TaskHive. Testers validated core features such as:

- **User Authentication:** Login and registration for clients, freelancers, and admins.
- **Job Management:** Creating, editing, viewing, and deleting jobs; bidding by freelancers.
- **Project Management:** Tracking deadlines, deliverables, and project progress.
- **Payments:** Processing payments and withdrawal requests.
- **Messaging & Notifications:** Sending and receiving real-time messages and alerts.
- **Admin Controls:** Managing users, jobs, disputes, and approvals.

Manual testing also focused on edge cases like invalid inputs, missing fields, and network interruptions, ensuring robustness and proper error handling. Interface responsiveness was verified across devices and browsers to maintain a consistent experience.

6.1.1 System Testing

System testing was conducted after integrating all modules to verify TaskHive’s overall behavior, focusing on performance, reliability, and interactions between users, projects, payments, and messaging.

Table 6.1.1.1: System Testing Table

<i>Test ID</i>	<i>Feature Tested</i>	<i>Description</i>	<i>Expected Result</i>	<i>Actual Result</i>	<i>Status</i>
<i>ST-01</i>	<i>User Authentication</i>	<i>Test login/signup for clients, freelancers, admin</i>	<i>Successful login/signup</i>	<i>As expected</i>	<i>Pass</i>
<i>ST-02</i>	<i>Job Posting</i>	<i>Client creates a new job</i>	<i>Job saved and visible in dashboard</i>	<i>As expected</i>	<i>Pass</i>
<i>ST-03</i>	<i>Bid Submission</i>	<i>Freelancer submits a bid for a job</i>	<i>Bid successfully stored</i>	<i>As expected</i>	<i>Pass</i>
<i>ST-04</i>	<i>Project Creation</i>	<i>Accepted bid creates a project</i>	<i>Project initialized with deadlines and progress</i>	<i>As expected</i>	<i>Pass</i>
<i>ST-05</i>	<i>Payment Processing</i>	<i>Client pays for a project milestone</i>	<i>Payment completed and recorded</i>	<i>As expected</i>	<i>Pass</i>

Table 6.1.1.1 summarizes the overall system testing results. System testing confirms that all major components, including authentication, job management, project workflow, payments, and messaging, perform reliably under test conditions.

The “Expected Result” defines how the system should ideally behave, while the “Actual Result” records the real output during testing. The “Status” column indicates whether the system passed or failed the test. The consistent “Pass” results demonstrate that all major components of the platform are stable and reliable under test conditions.

6.1.2 Unit Testing

Unit testing verified the functionality of individual modules in isolation before integration. Critical modules tested included:

- **Authentication Module:** Validates login credentials and role-based access.
- **Job Module:** Ensures creation, editing, deletion, and retrieval of job data.
- **Bid Module:** Validates submission, update, and acceptance of bids.
- **Project Module:** Checks tracking of progress, deadlines, and deliverables.
- **Payment Module:** Verifies processing of payments and withdrawal requests.
- **Messaging Module:** Tests sending and receiving messages within projects.
- **Notification Module:** Confirms alerts are generated and received correctly

Table 6.1.2.1: Unit Function Table

Module	Function	Expected Output	Actual Output	Status
Authentication	validateUser()	Valid credentials return success	Returned success	Pass
Job Management	createJob()	Job stored in database	Stored successfully	Pass

Bid Module	submitBid()	Bid saved and linked to job	Saved successfully	Pass
Project Module	createProject()	Project initialized with deadlines	As expected	Pass
Payment Module	processPayment()	Payment recorded and status updated	As expected	Pass
Messaging	sendMessage()	Message stored and received by recipient	As expected	Pass
Notification	sendNotification()	Notification delivered to user	As expected	Pass

Table 6.1.2.1: provides a breakdown of the unit testing phase. Each row represents a specific function or method tested within the system. The “Expected Output” column defines what the function was intended to return, while the “Actual Output” shows the observed behavior during testing. Since all outputs matched their expectations, the unit tests were marked as successful. This validation ensured that each isolated component worked correctly before integration with other modules, reducing the likelihood of system-level failures later.

6.1.3 Functional Testing

Functional testing verified that all use cases from TaskHive’s UML diagrams performed as intended for both clients and freelancers.

Table 6.1.3.1: Test Case Table

Test Case ID	Use Case	Input	Expected Output	Actual Output
TC-01	Register / Login	Valid credentials	Redirect to dashboard	Successful login
TC-02	Post Job	Job details entered	Job visible in job listings	As expected
TC-03	Submit Bid	Bid details entered	Bid linked to job	As expected
TC-04	Create Project	Accept bid	Project initialized	As expected
TC-05	Make Payment	Payment info entered	Payment processed and recorded	As expected
TC-06	Messaging	Send message	Message received by recipient	As expected
TC-07	Notifications	Trigger event	Notification delivered	As expected

Table 6.1.3.1: presents test cases for major use cases of the system. Each “Test Case ID” corresponds to a specific user or admin function derived from the UML use case diagram The “Input” column lists the data or action performed by the tester, while “Expected Output” and “Actual Output” detail what the system should and did return, respectively. The successful

“Pass” status across all test cases confirms that the TaskHive platform fulfills its intended functional requirements and offers a reliable workflow for users and administrators alike.

6.1.4 Integration Testing

Integration testing validated the interaction and data flow between interconnected modules.

Tested Integrations:

- User ↔ Job & Bid modules
- Client ↔ Project & Payment modules
- Project ↔ Messaging & Notification system
- Admin ↔ User & Job management modules

Table 6.1.4.1: Integration Testing Table

Integration ID	Modules Integrated	Expected Result	Actual Result	Status
IN-01	User & Job Module	Jobs and bids displayed correctly	As expected	Pass
IN-02	Client & Project Module	Project created and tracked correctly	As expected	Pass
IN-03	Project & Payment Module	Payment processed and recorded	As expected	Pass
IN-04	Messaging & Notification Module	Messages and notifications sent in real-time	As expected	Pass

IN-05	Admin & User/Job Management	Admin can view and manage users/jobs	As expected	Pass
-------	-----------------------------	--------------------------------------	-------------	------

Table 6.1.4.1: Documents the results of integration testing. Integration testing confirmed that **modules communicate correctly**, ensuring seamless operation across user, project, payment, messaging, and admin workflows.

6.2 Tools and Technologies

During the testing and evaluation phases of TaskHive, a set of tools and frameworks were employed to ensure that the platform performed efficiently, securely, and reliably. Each tool contributed to developing, testing, or validating specific system functionalities. Frontend and backend frameworks ensured smooth user interaction and data processing, while database and real-time communication technologies supported efficient job management, messaging, and notifications. Testing tools like Postman and GitHub facilitated quality assurance, version control, and collaborative development throughout the project.

Table 6.2.1: Tools and Technologies Table

Tool / Framework	Category	Purpose / Description
React.js	Frontend Framework	Used to create an interactive, responsive, and dynamic user interface for clients, freelancers, and admin dashboards.
Node.js	Backend Runtime Environment	Handles server-side logic, API requests, and communication between frontend and database.

Express.js	Web Framework	Provides backend routing, middleware support, and REST API handling for jobs, bids, projects, and payments.
PostgreSQL	Database	Stores users, jobs, bids, projects, payments, messages, reviews, and notifications securely.
Prisma	ORM / Database Toolkit	Maps database tables to objects, enabling structured queries and data integrity.
Socket.IO / WebSockets	Real-Time Communication	Enables instant messaging and real-time notifications between clients and freelancers.
Postman	API Testing Tool	Used for testing and validating RESTful API endpoints to ensure accurate backend responses.
Jest / Manual Testing	Testing Framework	Performed unit, integration, and functional testing to verify module functionality and reliability.
Figma	UI/UX Design Tool	Assisted in designing mockups and maintaining a consistent visual structure across the platform.
Git & GitHub	Version Control	Managed source code versions, collaboration, and backups during development and testing.

Table 6.2.1: highlights the key **tools and technologies** used in both the development and testing of TaskHive. Each tool is mapped to its purpose, showing how the combination of frontend, backend, database, real-time, and testing technologies contributed to the platform’s overall functionality, reliability, and user experience. This demonstrates a modern, integrated technology stack supporting efficient development, secure data handling, and seamless collaboration for freelancers and clients.

6.3 Summary

The testing and evaluation phase successfully validated **TaskHive’s** reliability, usability, and overall efficiency. Through structured manual, unit, functional, and integration testing, all core functionalities—including user authentication, job posting, bidding, project tracking, payments, messaging, and notifications—were thoroughly verified against the specified requirements.

All major modules passed their respective test cases, and no critical bugs were detected. Minor interface or usability issues identified during manual testing were promptly resolved, ensuring a consistent and smooth experience across web and mobile interfaces.

Integration between modules—such as jobs ↔ bids ↔ projects ↔ payments ↔ messaging—worked seamlessly, confirming robust data flow, correct backend logic, and real-time synchronization.

Overall, the testing phase concluded that **TaskHive** is stable, scalable, and ready for deployment. The platform meets all project objectives, providing secure authentication, efficient freelance project management, reliable payment processing, real-time communication, and a user-friendly interface for clients, freelancers, and administrators.

Chapter 7

CONCLUSION AND FUTURE WORK

7 Conclusion

The development of TaskHive successfully achieved its primary goal of designing and implementing a modern, secure, and user-friendly freelancing platform tailored specifically for Pakistani clients and freelancers. The system provides an integrated environment for job posting, bidding, project management, messaging, and reviews, ensuring transparency and efficiency throughout the freelancing process.

A key focus of TaskHive is addressing the limitations faced by Pakistani users on existing international platforms. By offering reduced commission rates, prioritizing local clients and freelancers, and planning future integration with Pakistani payment methods such as JazzCash and local banks, TaskHive presents a more accessible and fair freelancing ecosystem. These features help improve financial convenience, visibility, and trust for local users.

Throughout the project, strong emphasis was placed on usability, functionality, and scalability. TaskHive's modular architecture allows each component—ranging from authentication and project management to payments and communication—to operate independently while integrating smoothly with other modules. This design ensures ease of testing, maintenance, and future expansion.

The user interface, developed using React.js, delivers an intuitive experience for clients, freelancers, and administrators. The Node.js and PostgreSQL backend, combined with Socket.IO for real-time communication, ensures secure, fast, and reliable data handling. Comprehensive testing, including manual, unit, functional, and integration testing, confirmed that all modules functioned correctly and met the defined requirements.

In conclusion, TaskHive demonstrates how modern web technologies can be adapted to meet local economic and technical needs. The platform not only streamlines freelance collaboration but also lays a strong foundation for the growth of Pakistan's digital freelancing ecosystem and future enhancements in freelance technology.

7.1 Future Work

While the current version of TaskHive fulfills its core objectives, there are several opportunities for improvement and expansion:

1. **Mobile Application Development:**

Creating dedicated Android and iOS applications to make TaskHive accessible on mobile devices, enhancing convenience and user engagement.

2. **Localized Payment System Integration:**

Future versions of TaskHive will integrate Pakistani payment methods such as JazzCash, EasyPaisa, and local bank transfers. This will allow freelancers to withdraw earnings easily in local currency and enable clients to make payments without relying on international gateways.

3. **Reduced and Flexible Commission Model:**

The platform can further optimize its commission structure by introducing **flexible commission slabs** for long-term freelancers, verified users, or high-volume projects. This will encourage retention and provide better financial benefits to Pakistani freelancers.

4. **Escrow-Based and Milestone Payment System:**

TaskHive can implement an **escrow-based payment mechanism** where client payments are securely held and released automatically upon milestone completion. This will improve trust, reduce disputes, and ensure fair compensation for freelancers.

5. **Enhanced Admin Controls:**

Future improvements include advanced admin controls for better **dispute handling, fraud prevention, and compliance with local regulations**. Detailed analytics dashboards can help admins monitor platform activity, payments, and user behavior more effectively.

Chapter 8

REFERENCE

8 References

- [1] D. Flanagan, *JavaScript: The Definitive Guide*, 7th ed. Sebastopol, CA, USA: O'Reilly Media, 2020. [Online]. Available: <https://www.oreilly.com/library/view/javascript-the-definitive/9781491952023/>
- [2] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing React Apps*. Sebastopol, CA, USA: O'Reilly Media, 2017. [Online]. Available: <https://www.oreilly.com/library/view/learning-react-2nd/9781492051718/>
- [3] T. Holowaychuk, *Express.js Guide: The Comprehensive Book on Express.js*. Leanpub, 2019. [Online]. Available: <https://leanpub.com/expressjs>
- [4] PostgreSQL Global Development Group, *PostgreSQL Documentation*, 2024. [Online]. Available: <https://www.postgresql.org/docs/>
- [5] Node.js Foundation, *Node.js Documentation*, 2024. [Online]. Available: <https://nodejs.org/en/docs>
- [6] Socket.IO, *Socket.IO Documentation*, 2024. [Online]. Available: <https://socket.io/docs/>
- [7] Postman Inc., *Postman API Platform Documentation*, 2024. [Online]. Available: <https://learning.postman.com/>
- [8] GitHub, *GitHub Documentation*, 2024. [Online]. Available: <https://docs.github.com/>
- [9] Jest Community, *Jest Testing Framework Documentation*, 2024. [Online]. Available: <https://jestjs.io/docs>
- [10] Figma, *Figma Design System Documentation*, 2024. [Online]. Available: <https://help.figma.com/>
- [11] World Wide Web Consortium (W3C), *HTML & CSS Standards*, 2024. [Online]. Available: <https://www.w3.org/TR/>
- [12] UML.org, *Unified Modeling Language (UML) Specifications*, 2024. [Online]. Available: <https://www.uml.org/>
- [13] Stack Overflow, *Developer Documentation and Technical Discussions*, 2024. [Online]. Available: <https://stackoverflow.com/>